A Design Based Estimator for Regression Trees

Estimation of Population Trees Built from Sample Data

Pablo Parás Ochoa

Master's Degree Project in
Data Science, Statistics and Decision Analysis
Spring term 2026
Supervisor: Edgar Bueno Castellanos
Department of Computer
and Systems Sciences



Stockholm
University

Abstract

This thesis proposes a design based estimator that enables the estimation of the population regression tree from a sample. It is also concerned with the properties of this estimator, particularly in its ability to cover the population tree. To accomplish this, it also proposes a way to conceptualize coverage for regression trees and uses it in conjunction with the proposed estimator. The primary research question guiding this thesis is: How can regression trees be estimated from a sample as finite-population objects under design-based inference? The thesis proposes a point estimator and an interval estimator for the splits used to construct a tree. The estimators are evaluated through Monte Carlo simulation using the Boston Housing dataset to evaluate both split and region (tree) coverage. For this purpose, it draws 1000 Poisson samples at different expected sample sizes (50, 100, 150 and 200) to evaluate the performance of the estimator. The results indicate that, while coverage of all of the splits proved elusive, the proposed estimator was still able to recover some aspects of the population tree's structure, particularly as sample size increased. This was especially true for region-based coverage, which improved substantially at larger sample sizes. The results obtained by this thesis reflect the difficulty in estimating tree based objects because of their instability and sensitivity to the data. They show that it is possible to incorporate information about the sampling design into CART algorithms and provide a tentative measure of coverage for tree objects. Much work still remains to be done in this area, but this thesis provides a starting point on which it is possible to build further research.

Keywords: Regression Trees, CART, Survey Estimators

Acknowledgements

Enormous thanks to Anna Gabriella Cavagliano Martinez, my smart and lovely wife, who kept me from going insane while writing this. Thanks also go to my father, Pablo Gerardo Paras, who introduced me to and first taught me about survey methodology. Finally, considerable thanks also goes to my supervisor, Edgar Bueno Castellanos, whose role was vital in producing this thesis.

Abbreviations

Abbreviation	Description
CART	Classification and Regression Trees
i.i.d.	Independent and Identically Distributed
RSS	Residual Sum of Squares
CDF	Cumulative Distribution Function

List of Figures

7.1 Coverage rate by population split order for the estimator across sample sizes. 40
7.2 Average confidence interval width by population split order for the estimator across sample sizes. 41
7.3 Overall tree coverage rates across sample sizes for the estimator. 42

List of Tables

7.1 Example split-level inference results from a single simulation replicate. 36
7.2 Split-level performance metrics for the estimator. 39
7.3 Overall coverage metrics for the estimator. 40

Contents

Abstract **iii**

Acknowledgements **v**

Abbreviations **vii**

List of Figures **ix**

List of Tables **xi**

Contents **xiii**

1 Introduction **1**
1.1 Research Problem and Questions 3

2 Extended Background **5**
2.1 Scientific Grounding 5
2.2 Contribution to the Literature 6
2.3 Potential Applications of the Method Proposed 7

3 Regression Trees from the Entire Population **9**

4 Estimating the Population Tree from a Sample **15**
4.1 Point Estimation 15
4.2 Interval Estimation 18
4.2.1 An Algorithm for Measuring Uncertainty Around Split Selection 18
4.2.2 Conceptualizing Uncertainty in the Estimation of $\hat{M}$. 21
4.2.3 Comparing $\hat{M}$ to M 23

5 Method for Evaluation **27**

6 Ethical Considerations **31**
6.1 Method Alternatives and Ethical Considerations 31
6.2 Technology and AI usage 33

7 Results **35**
7.1 The Population Tree M 35
7.2 Example with One Estimated Tree 36
7.3 Coverage Results for the $\hat{M}$ Estimator 38

8 Discussion **43**
8.1 Analysis of the Results 43
8.2 Strengths and Limitations of the Study 45
8.3 Future Areas for Research 47
8.4 Conclusion 48

Bibliography **53**

Appendices **55**

1

Introduction

Since their introduction, tree based methods (e.g. regression trees, classification trees, random forests) have become indispensable tools in a data scientist's or statistician's toolkit. They enable the analysis of a wide variety of data types and structures in a flexible way which produces highly interpretable results. In an age of powerful black box models, this interpretability is key as it allows decision makers to understand the model and make informed decisions. Of particular interest to this thesis are regression trees, which can be used to construct highly flexible and interpretable regression models which make very few assumptions about the underlying distribution of the data or the relationship between explanatory and explained variables.

Despite these advantages, tree based methods are not naturally suited for working with data generated through sampling processes. Typical tree based methods usually assume that data is independently drawn and distributed (i.i.d.). In contrast, most modern sampling designs are complex in ways that do not produce i.i.d. data. These methods provide valuable insights into populations, but they also pose unique methodological challenges due to features such as unequal selection probabilities, clustering, and stratification. Recent academic work has sought to adapt tree based methods to work with survey data, though these approaches have primarily treated tree based methods as automated post-stratification techniques or as models for estimating population parameters [2] [14]. Regression trees themselves have not yet been treated directly as objects of inference.

This creates an important methodological gap. There is currently no established framework for estimating regression trees as finite-population quantities under a design-based perspective. Likewise, no standard framework exists for assessing the variability of such estimators. This thesis addresses this gap by developing point and interval estimators for regression trees from survey samples following a design-based approach. In doing so, it conceptualizes the regression tree itself as a population parameter and extends design-based inference to complex, structured objects.

In order to build this estimator, the thesis adopts the well known Horvitz-Thompson estimator for totals. It uses this estimator as a starting point to build a weighted residual sum of squares (RSS) with which it is possible to derive regression trees. In this way, it becomes possible to account for the sampling design in the construction of the regression tree, leading to a tree that better reflects the underlying population structure. This thesis also explores the concept of coverage of the population tree by an estimated tree derived from a sample.

The proposed methods, while imperfect, introduce a framework for estimating regression trees themselves as objects of inference, which has not been addressed in prior work. The use of tree based estimators with survey data could increase the tools available to statisticians working in survey contexts. Additionally, these methods could allow for the construction of regression trees in contexts where the use of the entire dataset is computationally expensive or impractical. The thesis explores and adapts techniques in innovative ways and contributes to the broader literature on explainable machine learning and survey estimation.

This thesis is structured as follows. First, a brief section formally presents both the central problem that motivates this thesis as well as the specific research question that drives the thesis. Afterwards, the Extended Background section presents an overview of the relevant literature which highlights both the recent advancements in the field of tree methods within survey contexts as well as the gaps within this literature that the thesis attempts to address. Chapters 3, 4 and 5 present the methods used for deriving the estimator for regression trees from both the Horvitz-Thompson estimator and commonly used methods for building regression trees. In addition, they examine the potential properties of this estimator and establish the methods used to evaluate it. The thesis also discusses method alternatives and ethical considerations in Chapter 6. The results of this evaluation are then presented in Chapter 7. Finally, the thesis presents a

discussion of the method proposed, its strengths and weaknesses and further avenues for research in Chapter 8.

1.1 Research Problem and Questions

While regression trees are widely used, the non-i.i.d. nature of survey data breaks with one of the main assumptions of the CART algorithm. As such, there is need for an estimator that can handle non-i.i.d. data to build regression trees from samples. There is also no clear way to conceptualize or measure the properties of that estimator. This is particularly important because it also means there is no way to quantify how much uncertainty exists around the estimator and how sensitive it is to sampling design, both of which would be necessary before any such method could be used and adopted in practice.

The problem at the heart of this thesis, therefore, is that no design-based framework exists for estimating regression trees as finite-population objects. As such, the goal of this study is to produce an estimator for the regression tree as a population quantity, evaluate it and explore some of its properties. The central question guiding this thesis therefore is: How can the regression tree obtained from a finite population be estimated from a sample under design-based inference?

2

Extended Background

2.1 Scientific Grounding

The CART algorithm, which is used to construct regression trees, works by recursively partitioning data into subsets or nodes and automatically splitting data into these nodes in such a way that they are as homogeneous as possible [3]. It has been expanded considerably since its creation to increase its accuracy and reduce variance, though often at the cost of interpretability [1]. It has also been expanded into Bayesian contexts [6].

In the world of surveys and survey methods, tree based methods have not yet been widely adopted [14]. Recently, methods have been developed for generating design based regression trees [13] and random forests [5]. These methods have been used for a wide variety of tasks related to surveys, such as non-response analysis and adjustments, creating pseudo-weights for non-probability samples, inference, modeling respondent burden, and perceived burden and even predicting survey costs [2][4][8][15]. Other uses have included the imputation of missing data in contexts where other methods may be less useful, such as when large numbers of auxiliary information is missing, modeling heterogeneous treatment effects in survey experiments and analyzing establishment non-response [7][9][11].

Recently, there have been multiple significant advancements in the development of consistent regression trees. A consistent regression tree is defined as an algorithmic model whose predictions converge to the true, underlying regression function as the sample size used to construct the tree

approaches infinity. “Naive” trees (which do not incorporate survey designs) are biased [10]. Different methods have been developed to account for design effects and incorporate sampling weights in the creation of tree models. These include: using weighted variance estimators as splitting criteria, calculating terminal node predictions as survey-weighted averages, and using regression trees conceptually as automated post-stratification methods [2][10][14].

While the methods mentioned so far have been developed for incorporating survey design in regression trees, they have primarily been used for model assisted estimation of population parameters [2][5][8] and small area estimation [2][14] both of which are outside the scope of this thesis. In addition to this, existing literature has used regression trees as estimators of superpopulation regression functions under complex sampling. In this context, trees are treated as random objects and the authors propose a measure of uncertainty over tree objects. Nevertheless, they work within a superpopulation and Bayesian model in which the source of the randomness is model based. The trees are then used to approximate regression functions [14].

Recent research has therefore sought to adapt tree based methods to survey contexts in three distinct ways. First, design features have been incorporated by modifying splitting criteria (e.g., weighted variance), using weighted predictions within terminal nodes, or interpreting trees as post-stratification tools [2][10]. Second, approaches differ in whether they adopt a model-based or design-based perspective. Bayesian regression trees, for instance, treat trees as random objects and model uncertainty through prior distributions [14]. In contrast to this, design-based approaches treat randomness as arising purely from the sampling design and the sample. Third, most existing work focuses on estimating population parameters, such as means or totals. This is done by constructing regression trees from sample data and using them in conjunction with auxiliary data to estimate population parameters [5][8]. This has also been done for small area estimation [14].

2.2 Contribution to the Literature

While these developments represent important progress, they leave a notable gap. In particular, existing methods largely treat regression trees as tools for estimating population parameters rather than as objects of inference in their own right. That is, the tree is typically used to assist in the

estimation of a parameter rather than being the parameter of interest being estimated. As a result, there is currently no well-developed framework for conducting design-based inference on the structure of regression trees themselves.

The thesis assumes fixed exogenous parameters and deterministic tie-breaking rules, which ensures that a unique regression tree can be associated with each population. Building on prior work, it extends the design-based perspective beyond estimating population parameters to the estimation of complex, structured objects. Specifically, the thesis formalizes the regression tree as a finite-population parameter and develops a framework for its estimation under complex sampling designs. In contrast to model-based approaches, in which randomness comes from the data-generating process, this study treats the population as fixed and randomness as arising solely from the sampling design.

This perspective introduces both conceptual and methodological challenges in estimating regression trees, which are addressed in in Chapter 4. Building on this formulation, the thesis contributes to the literature in two main ways. First, it develops a point estimator for the finite-population regression tree under general probability sampling designs. Second, it proposes an interval estimator, or more precisely, a region estimator, which is used to assess regional coverage of the population tree. This framework approximates uncertainty around split points and induced regions under repeated sampling by introducing the interval estimator, which enables an assessment of the reliability of the estimated regression trees.

In doing so, the thesis extends the scope of design-based inference beyond traditional parameters to complex structured objects. Through this, it hopes to contribute to both survey methodology and the broader literature on interpretable machine learning for non-i.i.d. data.

2.3 Potential Applications of the Method Proposed

This method could potentially have many applications both within survey statistics and in other fields. The clearest and most obvious application would be as a tool to understand complex relationships within a surveyed population without needing to rely on model assumptions. Regression trees are able to approximate complex non-linear relationships which may exist within the data in a highly transparent and interpretable fashion. Therefore, it may supplement traditional tools like linear regressions

within the field of survey statistics to explore an analyze information obtained from complex samples.

In addition to this, an estimator for population regression trees could also be used for the identification and analysis of heterogeneous subpopulations. By constructing regression trees that incorporate the sampling design into their construction, analysts can identify meaningful partitions of the population that reflect true structural differences instead of partitions that reflect noise generated through the sampling process. This could improve targeting in policy design, resource allocation, and program evaluation, especially where subgroup behavior is important.

As discussed previously, another potential application within the field is in non-response adjustment and calibration. A design-based regression tree could be used to define adjustment cells that are both data-driven and design-consistent. This could have advantages over traditional post-stratification, such as capturing higher-order interactions and nonlinearities more effectively.

Beyond survey methodology, the ideas and estimators proposed in this thesis could also contribute to the broader literature on interpretable machine learning for data generated through complex processes. Many datasets are obtained through non-i.i.d. processes, including clustered, stratified, or otherwise dependent structures. Extending tree based methods to handle such complexities within data generation and collection in a design-based manner may therefore have implications for other fields where such data generation processes exist.

Finally, the estimator could also be used in instances where constructing a regression tree from a full dataset is computationally expensive or impractical. In this way, regression trees could be constructed from samples obtained from the full dataset and be used to obtain inferences about relationships present in the full population. This is particularly relevant in modern data settings characterized by large-scale or streaming data, where using the entire dataset may be infeasible. In such contexts, the proposed estimator enables a trade off between computational efficiency and statistical precision. In particular, the coverage measures proposed by this paper can provide information on the reliability of the resulting partitions. This can be used to assess whether observed patterns are likely to persist at the population level when using a sample of the data.

3

Regression Trees from the Entire Population

This thesis constructs an estimator for M, the regression tree that would be obtained from a population U by using the CART algorithm. The proposed estimator is presented in a general way, in the sense that it can be used with any given sampling design. However, before it is possible to estimate a population quantity using a sample, it is first necessary to clearly define what is being estimated. As such, it is important to define both what this thesis considers to be M as well as how it can be obtained from the population. This section therefore formalizes how a regression tree for a given population is constructed.

To begin, it is important to note that a regression tree can be seen as an ordinary linear regression on a categorical variable in which this variable is dynamically constructed using the data. The goal, in this sense, is to construct a categorical variable whose categories are obtained by sequentially splitting already existing categories in such a way that the residual sum of squares (RSS) within each category is minimized at each step.

Let U be a finite population of size N. For each unit $i \in U$ let $y_i \in \mathbb{R}$ be the response variable of interest. Additionally, let $\mathbf{x}_i = (x_{i1}, \ldots, x_{ip}) \in \mathbb{R}^p$ be a vector of explanatory variables.

The regression tree M can therefore be defined as a partition of the predictor space $\mathcal{X} \subseteq \mathbb{R}^p$ into J regions $R_1, \ldots, R_J$, such that:

$$\bigcup_{j=1}^{J} R_j = \mathcal{X} \quad \text{and} \quad R_j \cap R_{j'} = \emptyset \text{ for } j \neq j'.$$

This partition is obtained by sequentially splitting the data (or a region obtained from a previous split) in two using one of the explanatory variables. The partitioning procedure continues sequentially until a predetermined stopping criterion is met. Generally, the best practice in the field is to first "grow" a tree until some arbitrary point typically related to node size or purity and then "prune" the tree using a measure of cost as related to complexity. However, for the purposes of this thesis, the stopping rule is a prespecified number of regions, J. Additionally, this thesis also assumes that a deterministic tie-breaking rule exists such that the tree construction procedure yields a unique partition for a given dataset. This is not unusual and is necessary to ensure that a single unique set of partitions, and therefore M, can be derived from a population.

Each split is defined by a pair (k, t), where $k \in \{1, \ldots, p\}$ indexes a explanatory variable and $t \in \mathbb{R}$ is the point within that variable chosen to produce the split. Each pair therefore produces a split in the data such that:

$$x_k \leq t \quad \text{and} \quad x_k > t.$$

At each step of the recursive procedure, the variable and split point are selected so as to minimize the RSS for the derived regions. The RSS can be defined as:

$$\mathrm{RSS} = \sum_{j=1}^{J} \sum_{i \in U : \mathbf{x}_i \in R_j} \left(y_i - \bar{y}_{R_j}\right)^2,$$

where

$$\bar{y}_{R_j} = \frac{1}{N_j} \sum_{i \in U : \mathbf{x}_i \in R_j} y_i, \quad N_j = \#\{i \in U : \mathbf{x}_i \in R_j\}.$$

The regression tree M therefore consists of three distinct elements:

- a set of splits $\{(k_\ell, t_\ell)\}_{\ell=1}^{L}$ which define the recursive partition (note that $L = J - 1$),
- a set of induced partitions or regions $\{R_1, \ldots, R_J\}$ of the predictor space (which is given by the set of splits as well as the child-parent relationships between the splits),

- the associated predictions $\bar{y}_{R_j}$ for each region.

These elements are, of course, intimately related and dependent on each other. However, they are not identical. Because of the hierarchical nature of the regression tree, the ordering and parent-child structure of the splits matter: an unordered collection of split rules does not uniquely determine the partition. As such, the set of splits must also include information about the recursive structure of the tree to be uniquely defined. Once the full tree structure is specified, however, the induced regions can be uniquely determined. At each stage, additional information (such as the selection of the region to split and the computation of within-region averages) is introduced that is not uniquely determined by the preceding elements.

In this sense, M can be considered to define a function $m : \mathcal{X} \to \mathbb{R}^p$ which is given by:

$$m(\mathbf{x}) = \bar{y}_{R_j} \quad \text{if } \mathbf{x} \in R_j.$$

For notational convenience, this thesis will represent regression trees based on the regions induced by the tree. Therefore, each tree can be understood as a set of regions bounded by a set of intervals using the explanatory variables. This thesis assumes that the explanatory variables are bounded by $(-\infty, \infty)$. Each terminal region R_j can therefore be expressed as the cartesian product of intervals:

$$R_j = \prod_{k=1}^{p} (a_{jk}, b_{jk}),$$

where $-\infty \leq a_{jk} < b_{jk} \leq \infty$ for all $k = 1, \ldots, p$.

As an example, consider a tree constructed using three variables x_1, x_2, and x_3. Suppose the root node is first split using $(x_1, 2.44)$. The left child node produced by this split is then split using $(x_2, 3.58)$, while the right child node is split using $(x_3, 3.29)$. Finally, the left child of the $(x_2, 3.58)$ split is further split using $(x_1, 1.05)$. This tree can be represented by five sets of intervals denoting five regions:

$$
\begin{aligned}
R_1 &= \{-\infty < x_1 < 1.05\} \times \{-\infty < x_2 < 3.58\} \times \{-\infty < x_3 < \infty\}, \\
R_2 &= \{1.05 < x_1 < 2.44\} \times \{-\infty < x_2 < 3.58\} \times \{-\infty < x_3 < \infty\}, \\
R_3 &= \{-\infty < x_1 < 2.44\} \times \{3.58 < x_2 < \infty\} \times \{-\infty < x_3 < \infty\}, \\
R_4 &= \{2.44 < x_1 < \infty\} \times \{-\infty < x_2 < \infty\} \times \{-\infty < x_3 < 3.29\}, \\
R_5 &= \{2.44 < x_1 < \infty\} \times \{-\infty < x_2 < \infty\} \times \{3.29 < x_3 < \infty\}.
\end{aligned}
$$

Note that different hierarchical structures of splits may produce the same final partition of the predictor space. However, the representation based on splits and their parent-child relationships fully preserves the recursive structure used to construct the tree. Additionally, the representation based on regions focuses only on the final partition induced by the tree. Both representations will be used within this thesis in both the theoretical and empirical portions.

The full algorithm used to construct the tree M is presented in the box below, note that J is assumed to be predetermined:

Algorithm 1: Construction of the population regression tree M

1. Initialization Start with a single region $R_1 = \mathcal{X}$, which contains the entire population predictor space.

2. Recursive splitting For each current terminal region R, and for every candidate split (k, t) within R:

- Partition R into:

$$R_L = \{\mathbf{x} \in R : x_k \leq t\}, \qquad R_R = \{\mathbf{x} \in R : x_k > t\}.$$

- Compute the corresponding within-region means:

$$\bar{y}_{R_L} = \frac{1}{N_{R_L}} \sum_{i \in U : \mathbf{x}_i \in R_L} y_i, \qquad \bar{y}_{R_R} = \frac{1}{N_{R_R}} \sum_{i \in U : \mathbf{x}_i \in R_R} y_i.$$

- Compute the resulting within-region RSS:

$$\mathrm{RSS}(R_L, R_R) = \sum_{i \in U : \mathbf{x}_i \in R_L} (y_i - \bar{y}_{R_L})^2 + \sum_{i \in U : \mathbf{x}_i \in R_R} (y_i - \bar{y}_{R_R})^2.$$

3. Split selection Select the region R and split (k, t) that produce the greatest reduction in the partition RSS.

4. Update partition Replace the selected terminal region R with its two induced subregions:

$$R \to \{R_L, R_R\}.$$

5. Stopping rule Repeat Steps 2-4 until the partition contains J terminal regions.

6. Prediction assignment For each terminal region R_j, assign the prediction:

$$\bar{y}_{R_j} = \frac{1}{N_j} \sum_{i \in U : \mathbf{x}_i \in R_j} y_i.$$

7. Output Return the regression tree M defined by the recursive partition.

4

Estimating the Population Tree from a Sample

4.1 Point Estimation

As stated previously, the main objective of this thesis is to construct an estimator $\hat{M}$ which approximates the regression tree M using a sample $s \subseteq U$. The estimator proposed in this section works for any sample s as long as that sample is obtained using a probability sampling design. That is to say, every unit $i \in U$ should have a strictly positive inclusion probability such that $\pi_i > 0$ for all $i \in U$.

The estimator $\hat{M}$ will be constructed as a function of population totals. This is convenient, as it enables the use of the well known Horvitz-Thompson estimator. It should be noted that other estimators can be considered (e.g. GREG estimation, difference estimation) but for simplicity this thesis concentrates exclusively on the Horvitz-Thompson estimator. For any given population total $T_y = \sum_{i \in U} y_i$, the corresponding Horvitz-Thompson estimator is defined as:

$$\hat{T}_y = \sum_{i \in s} \frac{y_i}{\pi_i},$$

where π_i denotes the inclusion probability of every unit in the sample. An estimator for the mean of y, denoted by $\bar{y}$, is given by:

$$\hat{\bar{y}} = \frac{\hat{T}_y}{\hat{N}}.$$

where

$$\hat{N} = \sum_{i \in s} \frac{1}{\pi_i}.$$

Within the regression tree's partitions, for any region $\hat{R} \subseteq \mathcal{X}$, it is possible to estimate the population quantities:

$$N_{\hat{R}} = \sum_{i \in U} \mathbb{I}(\mathbf{x}_i \in \hat{R}), \quad T_{y,\hat{R}} = \sum_{i \in U} y_i \, \mathbb{I}(\mathbf{x}_i \in \hat{R}),$$

where $N_{\hat{R}}$ denotes the number of observations in a given region $\hat{R}$ and where $T_{y,\hat{R}}$ represents the population total for y within that region. These population quantities also have corresponding Horvitz-Thompson estimators:

$$\hat{N}_{\hat{R}} = \sum_{i \in s} \frac{\mathbb{I}(\mathbf{x}_i \in \hat{R})}{\pi_i}, \quad \hat{T}_{y,\hat{R}} = \sum_{i \in s} \frac{y_i \, \mathbb{I}(\mathbf{x}_i \in \hat{R})}{\pi_i}.$$

In a similar fashion as before, these can be used to construct the estimator for $\bar{y}$ for the region by using:

$$\hat{\bar{y}}_{\hat{R}} = \frac{\hat{T}_{y,\hat{R}}}{\hat{N}_{\hat{R}}}.$$

Using these estimators it is possible to construct a sample-based estimator for the population RSS. This is very similar to the way the RSS is obtained for the population but weighting by the inclusion probabilities π_i and using the estimator for the region mean $\hat{\bar{y}}_R$:

$$\widehat{\mathrm{RSS}} = \sum_{j=1}^{J} \sum_{i \in s : \mathbf{x}_i \in \hat{R}_j} \frac{\left(y_i - \hat{\bar{y}}_{\hat{R}_j}\right)^2}{\pi_i}.$$

The estimator $\hat{M}$ can be obtained by sequentially splitting the data and selecting at each step the split $(\hat{k}, \hat{t})$ that minimizes the obtained $\widehat{\mathrm{RSS}}$.

The resulting estimated tree $\hat{M}$ consists of:

- a set of estimated splits $\{(k_\ell, \hat{t}_\ell)\}_{\ell=1}^{L}$,
- an induced partition $\{\hat{R}_1, \ldots, \hat{R}_J\}$ of the predictor space which is given by the set of splits as well as the child-parent relationships between the splits,
- and associated predictions $\hat{\bar{y}}_{\hat{R}_j}$ for each region.

Formally, the estimator $\hat{M}$ can be expressed as the solution to the optimization problem:

$$\hat{M} = \arg\min_{T \in \mathcal{M}_J} \widehat{\mathrm{RSS}}(T),$$

where T denotes a candidate regression tree and $\mathcal{M}_J$ denotes the space of all regression trees with J terminal regions.

In practice, this optimization problem is approximated using a greedy recursive partitioning algorithm in which splits are selected sequentially to achieve the largest reduction in $\widehat{\mathrm{RSS}}$ at each step. The full algorithm used to construct the estimator $\hat{M}$ is presented in the box below:

Algorithm 2: Construction of $\hat{M}$ using Horvitz-Thompson estimation

1. Initialization Start with a single region $\hat{R}_1 = \mathcal{X}$ which encompasses the entire sample.
2. Recursive splitting For each current terminal region $\hat{R}$, and for every candidate split $(k, \hat{t})$:

- Partition $\hat{R}$ into:

$$\hat{R}_L = \{\mathbf{x} \in \hat{R} : x_k \leq \hat{t}\}, \quad \hat{R}_R = \{\mathbf{x} \in \hat{R} : x_k > \hat{t}\}.$$

- Compute the corresponding Horvitz-Thompson estimators $\hat{\bar{y}}_{\hat{R}_L}$ and $\hat{\bar{y}}_{\hat{R}_R}$ as well as the corresponding within-region residual sum of squares: $\widehat{\mathrm{RSS}}(\hat{R}_L, \hat{R}_R)$

3. Split selection Select the region $\hat{R}$ and split $(k, \hat{t})$ that minimize the resulting partition $\widehat{\mathrm{RSS}}$.
4. Update partition Replace region $\hat{R}$ with its two induced subregions:

$$\hat{R} \to \{\hat{R}_L, \hat{R}_R\}.$$

5. Stopping rule Repeat Steps 2-4 until the partition contains J terminal regions.
6. Prediction assignment For each terminal region $\hat{R}_j$, compute:

$$\hat{\bar{y}}_{\hat{R}_j} = \frac{\hat{T}_{y,\hat{R}_j}}{\hat{N}_{\hat{R}_j}}.$$

7. Output Return the estimated regression tree $\hat{M}$ defined by the partitions.

The estimation procedure defined in the previous section exploits the fact that all of the quantities used to construct the regression tree $\hat{M}$ can be expressed as functions of population totals and thus can be estimated using Horvitz-Thompson estimators. It is not uncommon within survey theory to rewrite population parameters of interest θ as a function of q population totals:

$$\theta = f(T_1, T_2, \ldots, T_q), \quad \text{where } T_j = \sum_{i \in U} y_{ji}.$$

An estimator for θ, $\hat{\theta}$ can be obtained by replacing each population total with its Horvitz-Thompson estimator:

$$\hat{\theta} = f(\hat{T}_1, \hat{T}_2, \ldots, \hat{T}_q), \quad \text{where } \hat{T}_j = \sum_{i \in s} \frac{y_{ji}}{\pi_i}.$$

This is, essentially, what has been done to estimate RSS through $\widehat{\mathrm{RSS}}$ in this thesis. It should be noted that $\hat{\theta}$ is not, in general, an unbiased estimator for θ but it is consistent, which means it converges to θ as the sample size increases. Furthermore, there is no closed form expression for the variance of $\hat{\theta}$ but it can be approximated through the Taylor linearization technique. As such, the thesis expects that the properties of $\widehat{\mathrm{RSS}}$ will be consistent with the properties of $\hat{\theta}$ outlined above. While each component used in constructing $\hat{M}$ is a function of population totals, the recursive and data adaptive nature of the CART algorithm implies that $\hat{M}$ itself cannot be written as an explicit function of a finite set of totals. It therefore does not necessarily have the properties of the quantities used to construct it.

4.2 Interval Estimation

This is why it is important to propose a way to quantify uncertainty in the estimation of $\hat{M}$. Doing so makes it possible to assess how sensitive the selected splits and induced partitions are to the particular sample obtained under the sampling design. This in turn provides an indication of the reliability of the estimated tree structure.

4.2.1 An Algorithm for Measuring Uncertainty Around Split Selection

A given split point in the regression tree $\hat{M}$ selects the predictor variable x_k with a corresponding split point at $\hat{t}$. In order to quantify the uncertainty

associated with $\hat{t}$, this thesis uses an analogous approach to one generally used for the estimation of a population's median value through a sample (see e.g. Särndal et al. 2003 [12]), which is a Woodruff confidence interval. By using the weighted cumulative distribution function (CDF) to obtain an estimated quantile, constructing confidence intervals and then inverting that function to obtain an estimate of the confidence interval around $\hat{t}$.

The CDF of the predictor variable x_k at a point $\hat{t} \in \mathbb{R}$ within the region $\hat{R}$ may be written as:

$$F_{k|\hat{R}}(\hat{t}) = \frac{T_{z_k,\hat{R}}(\hat{t})}{N_{\hat{R}}},$$

where

$$T_{z_k,\hat{R}}(\hat{t}) = \sum_{i \in U} \mathbb{I}(\mathbf{x}_i \in \hat{R})\, z_{ik}(\hat{t}) \qquad \text{and} \qquad z_{ik}(\hat{t}) = \begin{cases} 1 & \text{if } x_{ik} \leq \hat{t}, \\ 0 & \text{otherwise,} \end{cases}$$

and

$$U_{\hat{R}} = \{i \in U : \mathbf{x}_i \in \hat{R}\} \qquad \text{and} \qquad N_{\hat{R}} = \sum_{i \in U} \mathbb{I}(\mathbf{x}_i \in \hat{R}).$$

Here, $U_{\hat{R}}$ denotes the subpopulation associated with the region $\hat{R}$ and $N_{\hat{R}}$ its population size. Note that the interval estimation is always performed with respect to the subpopulation defined by $\hat{R}$, so the relevant population changes recursively as the tree is partitioned.

The conditional CDF of x_k given $\hat{R}$ evaluated at the population split point $\hat{t}$ is:

$$p_k = F_{k|\hat{R}}(\hat{t}),$$

An estimator of p_k can therefore be obtained using the Horvitz-Thompson estimator:

$$\hat{p}_k = \hat{F}_{k|\hat{R}}(\hat{t}) = \frac{\hat{T}_{z_k,\hat{R}}(\hat{t})}{\hat{N}_{\hat{R}}}, \quad \text{where} \quad \hat{T}_{z_k,\hat{R}}(\hat{t}) = \sum_{i \in s} \frac{\mathbb{I}(\mathbf{x}_i \in \hat{R})\, z_{ik}(\hat{t})}{\pi_i}, \quad \hat{N}_{\hat{R}} = \sum_{i \in s} \frac{\mathbb{I}(\mathbf{x}_i \in \hat{R})}{\pi_i}.$$

We can estimate the variance of the CDF at point $\hat{t}$ within region $\hat{R}$ by:

$$\widehat{\mathrm{Var}}\big(\hat{F}_{k|\hat{R}}(\hat{t})\big) = \frac{1}{\hat{N}^2_{\hat{R}}} \sum_{i\in\hat{R}} \sum_{j\in\hat{R}} \frac{\Delta_{ij}}{\pi_{ij}} \left(\frac{z_{ik}(\hat{t}) - \hat{p}_k}{\pi_i}\right) \left(\frac{z_{jk}(\hat{t}) - \hat{p}_k}{\pi_j}\right),$$

where $\Delta_{ij} = \pi_{ij} - \pi_i\pi_j$ and $\pi_{ij} = P(i \in s, j \in s)$.

An approximate simultaneous $100(1-\alpha)\%$ confidence interval for p_k can then be obtained through the expressions:

$$L_k = \hat{p}_k - \hat{t}_{1-\alpha/(2L),\,\nu}\sqrt{\widehat{\mathrm{Var}}\big(\hat{p}_k\big)}, \quad U_k = \hat{p}_k + \hat{t}_{1-\alpha/(2L),\,\nu}\sqrt{\widehat{\mathrm{Var}}\big(\hat{p}_k\big)}.$$

Here, $\hat{t}_{1-\alpha/(2L),\,\nu}$ is the $(1-\alpha/(2L))$ quantile of the Student t-distribution with ν degrees of freedom, and L denotes the number of split points for which confidence intervals are simultaneously constructed. The factor $\alpha/(2L)$ comes from the Bonferroni correction for multiple testing and is used because inference is performed simultaneously for multiple split points in the regression tree.

Finally, an approximate confidence interval for the split point can be obtained by inverting the estimated CDF so that it is defined by:

$$\left(\hat{F}^{-1}_{k|\hat{R}}(L_k),\ \hat{F}^{-1}_{k|\hat{R}}(U_k)\right),$$

where $\hat{F}^{-1}_{k|R}(p)$ is the weighted empirical quantile function of x_k at p within region $\hat{R}$.

This method can be used to construct an algorithm that can approximate confidence intervals around the splits for $\hat{M}$:

Algorithm 3: Confidence interval for a split point

Input: Sample $s \subseteq U$ with first and second-order inclusion probabilities $\{\pi_i, \pi_{ij}\}_{i,j \in s}$; region $\hat{R}$; split variable x_k with estimated split point $\hat{t}$.

Step 1 (Distribution estimation): Estimate the cumulative distribution function of x_k within region $\hat{R}$:

$$\hat{p}_k = \hat{F}_{k|\hat{R}}(\hat{t}) = \frac{\hat{T}_{z_k,\hat{R}}(\hat{t})}{\hat{N}_{\hat{R}}},$$

where

$$\hat{T}_{z_k,\hat{R}}(\hat{t}) = \sum_{i \in s} \frac{\mathbb{I}(\mathbf{x}_i \in \hat{R})\, \mathbb{I}(x_{ik} \leq \hat{t})}{\pi_i}, \qquad \hat{N}_{\hat{R}} = \sum_{i \in s} \frac{\mathbb{I}(\mathbf{x}_i \in \hat{R})}{\pi_i}.$$

Step 2 (Interval construction): Using $\widehat{\mathrm{Var}}(\hat{p}_k)$, construct an approximate $(1-\alpha)$ confidence interval $[L_k, U_k]$ for p_k, and define

$$\hat{t}_L = \hat{F}^{-1}_{k|\hat{R}}(L_k), \qquad \hat{t}_U = \hat{F}^{-1}_{k|\hat{R}}(U_k).$$

Output: The interval $[\hat{t}_L, \hat{t}_U]$ as an approximate confidence interval for the split point $\hat{t}$.

4.2.2 Conceptualizing Uncertainty in the Estimation of $\hat{M}$

There is no widely established way to measure the distance between two regression trees. As previously explained, a regression tree can be thought as consisting of three principal components: first the set of splits, second the partition of the data induced by those splits, and third, the associated predictions for each of the regions. Additionally, the hierarchical structure of the splits is itself a distinguishing feature between trees. Because regression trees are structured objects, it is difficult to define an "average" tree or a corresponding measure of distance between trees. This in turn makes it difficult to characterize uncertainty in entire tree structures.

These difficulties mean it is simpler to consider uncertainty in the individual components of a tree instead of the entire tree. Each of these components may therefore differ across repeated realizations of the estimator $\hat{M}$. Similarly, the quantities used to construct $\hat{M}$ such as $\widehat{\mathrm{RSS}}$ are also subject to sampling variability. As a result, the quantities used to construct $\hat{M}$ and the components of the resulting tree can all be candidates for approximate measures of uncertainty. Accordingly, uncertainty in $\hat{M}$ may arise

from changes in the selected splits, the induced partitions, and the predicted values within regions.

In this sense, the thesis proposes an approximate way to characterize uncertainty around split selection through interval estimation. The objective is not to obtain exact distributional results for the split points, but rather to construct intervals that provide information about how sensitive the selected splits are to repeated sampling. Other alternatives exist, such as studying uncertainty in the induced partitions or in the predicted values themselves, though these are not explored in this thesis.

To accomplish this, the estimated split point obtained by $\hat{M}$ is related to its empirical position within the ordered values of the corresponding predictor variable x_k. This makes it possible to use established techniques for quantile estimation and quantile uncertainty under probability sampling designs. This interpretation is justified by the fact that candidate split points are selected from observed values of x_k, making their position inherently tied to the empirical distribution of the predictor.

The thesis exploits the fact that quantiles are the inverse of the CDF evaluated at an observed probability level. From this perspective, the split point $\hat{t}$ may be written approximately as:

$$\hat{t} = \hat{F}^{-1}_{k|\hat{R}}(\hat{p}_k),$$

where $\hat{F}^{-1}_{k|\hat{R}}$ denotes the weighted empirical quantile function of x_k within the estimated region $\hat{R}$, and $\hat{p}_k$ represents the corresponding estimated quantile level.

This emphasizes that the split is determined using the observed distribution of x_k within the region. Equivalently, $\hat{t}$ can be seen as the minimum of a stepwise "objective function" defined over the ordered values of x_k. In this view, estimating $\hat{t}$ reduces to selecting an order statistic indexed by the empirical distribution. When the population objective function (RSS as a function of t) has a clear minimum and x_k is locally smooth around the true split, the uncertainty of $\hat{t}$ largely reflects fluctuations of the empirical distribution around t.

This perspective allows the uncertainty of $\hat{t}$ to be approximated using standard quantile estimation results. The resulting confidence intervals are first-order approximations that capture uncertainty from estimating the underlying distribution but do not fully account for additional uncertainty arising from the recursive split selection procedure.

This perspective also highlights conditions under which this approximation is expected to perform well. In particular, the method is likely to be more reliable when there is sufficient data around the population split point t and when the population objective function has a clearly defined minimum. In contrast, when this objective function is relatively flat then the uncertainty of $\hat{t}$ may be underestimated by this approximation.

4.2.3 Comparing $\hat{M}$ to M

Once the split points and their corresponding confidence intervals have been obtained, it is necessary to define a practical way to actually compare the estimated tree $\hat{M}$ to the population tree M. The purpose of this comparison is to assess if the method proposed by this thesis produces trees $\hat{M}$ whose structure matches that of M.

The method proposed by this thesis is the following. First, the estimator $\hat{M}$ is constructed and confidence intervals for each split point are obtained through the methods established above. Then, the intervals are used to construct lower and upper bounds on the inequalities used to define each region. These bounds can be thought of as "confidence boxes", given that they are built from the confidence intervals and are used to represent uncertainty around the estimation of a given region. If the bounds derived in this fashion entirely contain those obtained from M then this thesis considers that the estimated region $\hat{R}$ covers the population region R. Consequently, the trees are evaluated through the regions induced by the splits.

For a confidence interval associated with a split point: $[\hat{a}_{jk}, \hat{b}_{jk}]$, the length of the confidence interval is defined as $\hat{b}_{jk} - \hat{a}_{jk}$. These interval lengths quantify the uncertainty associated with the estimated split points and determine the size of the corresponding confidence boxes.

More formally, the comparison between M and $\hat{M}$ can be expressed in terms of set inclusion between the population regions and the estimated regions. Recall that

$$R_j = \prod_{k=1}^{p} (a_{jk}, b_{jk}) \qquad \text{and} \qquad \hat{R}_j = \prod_{k=1}^{p} (\hat{a}_{jk}, \hat{b}_{jk}).$$

A population region is considered to be correctly covered if

$$R_j \subseteq \hat{R}_j.$$

As such, correct coverage of the j-th population region requires that:

$$\hat{a}_{jk} \leq a_{jk} \quad \text{and} \quad b_{jk} \leq \hat{b}_{jk} \qquad \text{for all } k = 1, \ldots, p.$$

For example, consider the population tree previously exemplified which uses explanatory three variables x_1, x_2, and x_3 in which the root node is first split using $(x_1, 2.44)$. The left child node produced by this split is then split using $(x_2, 3.58)$, while the right child node is split using $(x_3, 3.29)$. Finally, the left child of the $(x_2, 3.58)$ split is further split using $(x_1, 1.05)$. As was shown before, one of the resulting regions can be written as:

$$R_2 = \{1.05 < x_1 < 2.44\} \times \{-\infty < x_2 < 3.58\} \times \{-\infty < x_3 < \infty\}.$$

In the estimated tree $\hat{M}$, each split point is replaced by a confidence interval. Suppose, for instance, that the corresponding estimated splits for x_1 are given by the intervals $[0.95, 1.15]$ and $[2.30, 2.60]$, and for x_2 by $[3.40, 3.70]$. The analogous estimated region can then be written as:

$$\hat{R}_2 = \{0.95 < x_1 < 2.60\} \times \{-\infty < x_2 < 3.70\} \times \{-\infty < x_3 < \infty\}.$$

Note that for lower bounds of a region, the lower endpoint of the confidence interval is used, whereas for upper bounds, the upper endpoint is used instead. The comparison between M and $\hat{M}$ can then be carried out by checking whether the inequalities defining each region in M are contained within the corresponding inequalities in $\hat{M}$.

In the example above, to ensure the population tree is covered by $\hat{M}$ we must verify that:

$$R_1 \subseteq \hat{R}_1, \qquad R_2 \subseteq \hat{R}_2, \qquad R_3 \subseteq \hat{R}_3, \qquad R_4 \subseteq \hat{R}_4, \qquad R_5 \subseteq \hat{R}_5.$$

For example, for R_2, we verify that

$$0.95 = \hat{a}_{21} \leq a_{21} = 1.05, \qquad b_{21} = 2.44 \leq \hat{b}_{21} = 2.60,$$

and

$$-\infty = \hat{a}_{22} \leq a_{22} = -\infty, \qquad b_{22} = 3.58 \leq \hat{b}_{22} = 3.70,$$

and

$$-\infty = \hat{a}_{23} \leq a_{23} = -\infty, \qquad b_{23} = \infty \leq \hat{b}_{23} = \infty.$$

Since these inequalities hold, it follows that

$$R_2 \subseteq \hat{R}_2.$$

In practical terms, a population region is therefore considered to be correctly covered by the estimator if all of its bounds fall within the estimated intervals.

5

Method for Evaluation

This thesis uses a Monte Carlo simulation study to (empirically) calculate the actual confidence level achieved by the proposed method under a desired nominal confidence level of 95%. To do this, the thesis implements the proposed algorithm for estimating $\hat{M}$, as well as the method for approximating uncertainty around split selection, in `R`. The full code for the analysis is available in Appendix 1. Both are evaluated using the Boston Housing Dataset. This dataset is publicly available online; as such, there is no formal data collection methodology. However, some considerations were made when selecting the data set; these are explained in more detail in the next section.

To provide a point of comparison, the regression tree M was first constructed using the full dataset ($N = 506$). `MEDV` was used as the response variable y. This variable represents the median value of owner-occupied homes, measured in thousands of dollars. The remaining 13 variables were used as the set of explanatory variables $\mathbf{x}$.

Samples were then drawn using Poisson sampling (at different expected sizes) with the inclusion probabilities proportional to `MEDV`. Based on this sample, the estimator $\hat{M}$ was constructed using the estimator described in Section 4.1. The tree was built using $L = 5$. Approximate confidence intervals for the split points were then constructed using the method described in Section 4.2. Given that $L = 5$ the approximate confidence intervals were

constructed using $\alpha = 0.05/5$ as described in that section to apply the Bonferroni correction for multiple testing. Additionally, because the simulations use Poisson sampling, the general variance estimator was simplified to the Poisson variance estimator:

$$\widehat{\mathrm{Var}}(\hat{p}) = \frac{\sum_{i=1}^{n}(w_i^2 - w_i)(z_i - \hat{p})^2}{\hat{N}^2}$$

where

$$z_i = \mathbf{1}(x_i \leq t), \qquad \hat{N} = \sum_{i=1}^{n} w_i, \qquad \text{and} \qquad \hat{p} = \frac{\sum_{i=1}^{n} w_i z_i}{\hat{N}}.$$

To evaluate how well the estimated tree $\hat{M}$ approximates the population tree M, the following procedure was used for each generated sample.

First, a sample was selected from the population and used to construct a point estimate of the tree, denoted by $\hat{M}$. Next, confidence intervals were obtained for each estimated split point. These were used to construct estimated regions by replacing the split values in $\hat{M}$ with their corresponding interval bounds. In this way confidence intervals (or boxes in the case of regions) around the estimated splits and estimated regions were constructed.

The evaluation was then carried out at two different levels. First, each estimated split $\hat{t}$ was compared with its corresponding population split t. If the population split was contained within the confidence interval, that split was considered to be covered by the estimated tree $\hat{M}$. This allowed for a local assessment of the coverage of the splits and was used to evaluate the estimator's ability to correctly select splits.

Second, each population region R_j induced by M was compared against the corresponding estimated region $\hat{R}_j$. This comparison was used to assess whether each region in the estimated tree $\hat{M}$ contained a true population region. If all of the population regions were contained by a corresponding estimated region, the tree $\hat{M}$ was considered to cover the tree M. This provides a global assessment of whether the estimated tree structure successfully captures the overall partition induced by M.

The process was repeated by selecting 1000 samples for each expected sample size $n = 50, n = 100, n = 150$ and $n = 200$. For each sample, a corresponding tree $\hat{M}$ was constructed and approximate confidence intervals for the split points were obtained. The resulting intervals were then compared to the splits in M to assess empirical coverage and uncertainty across repeated samples as well as to obtain the average length (or size) of the intervals. The splits were evaluated in population tree order (to confirm

whether deeper splits were harder to cover). The most important metric, however, was regional coverage.

The results for this evaluation are presented in Chapter 7.

6

Ethical Considerations

6.1 Method Alternatives and Ethical Considerations

This thesis adopts an empirical, simulation-based research strategy to evaluate the proposed estimator for finite-population regression trees. The objective is methodological: to construct a point and a confidence interval estimator and assess their statistical properties. Of special interest is the ability of the estimator to cover the population tree and the uncertainty of its components under repeated sampling. Given the goals of this thesis, deriving the theoretical properties of the proposed estimator analytically would have been highly complex and they are considered beyond its scope. Therefore the thesis is limited to evaluate the proposal via a Monte Carlo simulation

The chosen approach is closely aligned with established practices in survey statistics, where repeated sampling from a known population is commonly used to study the finite-sample properties of estimators. By treating a complete dataset as a proxy for a finite population, it becomes possible to generate multiple samples under a specified probability sampling design and directly compare sample-based estimates to known population quantities. This enables the evaluation of properties such as bias, uncertainty, and coverage in a controlled setting. While a discussion of the bias of the estimator is outside the scope of this thesis, the proposed methodology could theoretically be extended to include it.

This contrasts with the Bayesian approaches used in recent tree based survey methods. These methods rely on a superpopulation model and treat

the regression tree as a random object, with uncertainty arising from an assumed data-generating process rather than from the sampling design [14]. Instead, this thesis adopts a strictly design-based perspective. This choice is a better match for the objective of isolating uncertainty induced by the sampling design rather than by model assumptions.

The choice of Poisson sampling is motivated by both theoretical and practical considerations. Poisson sampling is a well-understood probability sampling design with independent inclusion indicators. This simplifies both the implementation of the estimator and the estimation of uncertainty around split selection. The use of Poisson sampling therefore serves as a baseline case that facilitates clear evaluation without loss of generality.

The decision to base the estimator on Horvitz–Thompson estimators is similarly motivated. The Horvitz–Thompson estimator can be used to estimate population totals under general probability sampling designs unbiasedly and forms a cornerstone for design-based inference. More complicated estimators like generalized regression (GREG) estimators could potentially improve efficiency by incorporating auxiliary information. However, the primary objective of this thesis is to develop a clear way for estimating regression trees from a sample in a way that is generalizable. In that sense, simplicity was chosen over potentially more complicated methods which may improve efficiency. The Horvitz–Thompson approach therefore provides a natural and transparent starting point. Additionally, because the proposed estimator is based on Horvitz–Thompson estimation, it can easily be extended to more complex sampling designs.

Furthermore, the use of a greedy recursive partitioning algorithm (CART) is justified by both computational and conceptual considerations. While global optimization over the space of all possible trees could give a more optimal tree, it was computationally infeasible in practice. The greedy approach provides a tractable approximation that is widely used in both statistical and machine learning applications. By adapting this procedure to incorporate design-based estimators at each step, the thesis ensures consistency with established methodology while extending it to the survey sampling context. The use of greedy recursive partitioning is also consistent with the standard implementation of CART in classical statistical learning and aligns with its adaptations to survey data, where similar recursive splitting procedures are used with design-based modifications [2].

Regarding data selection, the Boston Housing Dataset was chosen due to its completeness and its availability online. In particular, it was important that the data contained a sufficient number of numerical variables so

that they could be used to build the regression tree. The relatively small size of the dataset was also important, as given the limited scope of a Master's level thesis using a smaller dataset helped save on computational resources and time. All data used by this thesis are publicly accessible and do not contain sensitive or personally identifiable information. As such, this thesis does not represent a risk to the privacy or confidentiality of the individuals in the data set. All analyses were conducted with an eye on maintaining ethical standards and the data were used in accordance with their terms of use and only for research purposes.

This thesis also emphasizes transparency and reproducibility. The methods and computational procedures are fully documented, and the R code used for simulation and estimator evaluation is available in Appendix 1. Finally, while this work is methodological in nature, caution is noted to avoid overstating conclusions or generalizing results beyond the scope of the research questions and the thesis.

6.2 Technology and AI usage

This thesis used R extensively to implement the methodology described above. This involved building functions that allowed for the construction of regression trees, computing Horvitz–Thompson estimates for means and residual sums of squares, and generating confidence intervals for tree splits. Additional functions were developed to simulate Poisson sampling, match estimated splits to population splits, and evaluate whether the resulting estimated regions contained the true population regions.

Simulations were then conducted at varying sample sizes to assess the coverage of confidence intervals and the accuracy of split estimation. The workflow utilized the Boston Housing Dataset from the MASS package, the tidyverse for data manipulation and piping, and purrr to efficiently handle iteration and mapping over simulations. This approach allowed for a comprehensive assessment of how sampling variability and weighting influence regression tree construction, split uncertainty, and region coverage.

AI was used primarily to compare thesis drafts with the thesis instructions and grading criteria to identify weak spots that were then addressed by the author. In addition, it was used to ensure that the mathematical notation developed by the author was consistent and without errors. Finally, while AI was not used to write code, it was used as an aid to identify bugs within the R code.

7

Results

Two approaches for the comparison of the population tree M and the estimated tree $\hat{M}$ were used. The first approach was comparison through the use of splits, where a given split in the population tree was considered covered if the estimated tree had a split with the same variable x_k where the confidence interval associated with the estimated split $\hat{t}$ contained the population split t. The second approach used involved using the confidence intervals associated with each split in $\hat{M}$ to construct region bounds and then comparing those bounds against the regions predicted by the population tree. If all population regions were contained by at least one estimated region then the population tree M was considered covered by the estimated tree $\hat{M}$.

This chapter first presents the population tree. It then presents the simulation results obtained using the weighted estimator for $\hat{M}$, including split-level coverage, interval widths, and overall tree coverage.

7.1 The Population Tree M

The population tree M was constructed using the method outlined in Chapter 3 with $L = 5$. This means that the tree contains five splits and six regions. The five splits defining the population tree are:

$$\begin{pmatrix} rm & lstat & rm & dis & rm \\ 6.939 & 14.370 & 7.420 & 1.357 & 6.540 \end{pmatrix}.$$

Given that for the Boston Housing dataset all $x_k \geq 0$, the regions produced by the tree are then:

$$R_1 = \{0 < rm < 6.939\} \times \{0 < lstat < 14.370\} \times \{0 < dis < 1.3567\},$$

$$R_2 = \{0 < rm < 6.540\} \times \{0 < lstat < 14.370\} \times \{dis \geq 1.3567\},$$

$$R_3 = \{6.540 \leq rm < 6.939\} \times \{0 < lstat < 14.370\} \times \{dis \geq 1.3567\},$$

$$R_4 = \{0 < rm < 6.939\} \times \{lstat \geq 14.370\},$$

$$R_5 = \{6.939 \leq rm < 7.420\},$$

$$R_6 = \{rm \geq 7.420\}.$$

Note that all variables present in the dataset which were not used to define a given region are taken to have the interval $\{-\infty \leq x_k \leq \infty\}$.

This is the set of splits and the set of regions against which the estimator $\hat{M}$ was compared during the simulations. For simplicity, it was obtained from the regression tree and then hard coded during the simulations to avoid manual recalculation with each simulation.

7.2 Example with One Estimated Tree

This section briefly shows, in practice, how the method works using a single simulated tree. The tree was specifically picked as an example because of how well it covered the population splits and regions and should not be taken as representative for the general results of the estimator. The splits and confidence intervals for the tree are presented in Table 7.1. The table reports the population split value, estimated split value, corresponding confidence interval, and whether the interval covered the population split, and is shown below:

STEP	VARIABLE	POPULATION	ESTIMATE	95% CI	COVERED
1	rm	6.9390	6.9390	[6.8260, 7.0790]	TRUE
2	lstat	14.3700	14.7000	[13.2700, 17.5800]	TRUE
3	rm	7.4200	7.4200	[7.3330, 7.6910]	TRUE
4	dis	1.3567	1.3567	[1.1691, 1.6180]	TRUE
5	rm	6.5400	6.5160	[6.3950, 6.6350]	TRUE

Table 7.1: Example split-level inference results from a single simulation replicate.

In the estimated tree $\hat{M}$, each split point is replaced by a confidence interval.

Estimated confidence regions (or "confidence boxes") were constructed by replacing each split threshold with the corresponding confidence interval endpoint that produced the widest admissible region. Upper bounds used the upper CI endpoint, while lower bounds used the lower CI endpoint. The resulting estimated regions are therefore:

$$\hat{R}_1 = \{0 < rm < 7.079\} \times \{0 < lstat < 17.580\} \times \{0 < dis < 1.6180\},$$

$$\hat{R}_2 = \{0 < rm < 6.635\} \times \{0 < lstat < 17.580\} \times \{dis \geq 1.1691\},$$

$$\hat{R}_3 = \{6.395 \leq rm < 7.079\} \times \{0 < lstat < 17.580\} \times \{dis \geq 1.1691\},$$

$$\hat{R}_4 = \{0 < rm < 7.079\} \times \{lstat \geq 13.270\},$$

$$\hat{R}_5 = \{6.826 \leq rm < 7.691\},$$

$$\hat{R}_6 = \{rm \geq 7.333\}.$$

The population regions can then be compared directly against the estimated confidence boxes. For example, for R_3:

$$R_3 = \{6.540 \leq rm < 6.939\} \times \{0 < lstat < 14.370\} \times \{dis \geq 1.3567\},$$

while the corresponding estimated region is

$$\hat{R}_3 = \{6.395 \leq rm < 7.079\} \times \{0 < lstat < 17.580\} \times \{dis \geq 1.1691\}.$$

Coverage is verified by checking whether all population bounds are contained within the estimated bounds:

$$6.395 = \hat{a}_{31} \leq a_{31} = 6.540, \qquad b_{31} = 6.939 \leq \hat{b}_{31} = 7.079,$$

$$0 = \hat{a}_{32} \leq a_{32} = 0, \qquad b_{32} = 14.370 \leq \hat{b}_{32} = 17.580,$$

and

$$1.1691 = \hat{a}_{33} \leq a_{33} = 1.3567, \qquad b_{33} = \infty \leq \hat{b}_{33} = \infty.$$

Since all inequalities hold, it follows that

$$R_3 \subseteq \hat{R}_3.$$

In this way we verify that:

$$R_1 \subseteq \hat{R}_1, \qquad R_2 \subseteq \hat{R}_2, \qquad R_3 \subseteq \hat{R}_3, \qquad R_4 \subseteq \hat{R}_4, \qquad R_5 \subseteq \hat{R}_5, \qquad R_6 \subseteq \hat{R}_6.$$

As can be seen, in practical terms, a population region is considered to be correctly covered whenever it is fully contained within the corresponding estimated "confidence box".

7.3 Coverage Results for the $\hat{M}$ Estimator

A total of 1000 Poisson samples were generated from the Boston Housing dataset using inclusion probabilities proportional to the response variable `MEDV`. Simulations were conducted for expected sample sizes of 50, 100, 150, and 200. Table 7.2 summarizes the split-level coverage rates and the corresponding average confidence interval widths for each population split. In the table, coverage rate denotes the proportion of simulations in which the confidence interval covered the population split value. Average width denotes the mean confidence interval width for each split.

Coverage rates differed substantially across the five population splits. The first split consistently obtained the highest coverage rates for smaller sample sizes, with coverage equal to 0.540 at $n = 50$. However, coverage for this split remained relatively stable across sample sizes and did not increase substantially beyond this level. In contrast, the second, third, and fourth splits showed clear improvements in coverage as the expected sample size increased. For example, coverage of the fourth split increased from 0.041 at $n = 50$ to 0.488 at $n = 200$. The fifth split remained difficult to cover across all sample sizes, with coverage rates remaining below 0.14 even for the largest sample size.

The average widths of the approximated confidence intervals decreased as the expected sample size increased, though there were a couple of small exceptions. This behavior is, for the most part, consistent with the expectations of the method used to construct the confidence interval which produces larger values as the sample size $\hat{N}_R$ decreases. For example, the average interval width for the second split decreased from 19.9 at $n = 50$ to 3.11 at $n = 200$, while the width for the fourth split decreased from 20.0 to 2.12 with that same increase in sample size. Despite this overall reduction

SAMPLE SIZE	POPULATION SPLIT	COVERAGE RATE	AVERAGE WIDTH
50	1	0.540	6.27
50	2	0.373	19.9
50	3	0.289	16.6
50	4	0.041	20.0
50	5	0.163	25.9
100	1	0.561	2.08
100	2	0.471	8.02
100	3	0.347	7.85
100	4	0.182	7.55
100	5	0.103	14.7
150	1	0.558	1.40
150	2	0.514	4.41
150	3	0.388	3.64
150	4	0.323	4.94
150	5	0.126	9.47
200	1	0.545	1.06
200	2	0.591	3.11
200	3	0.485	2.52
200	4	0.488	2.12
200	5	0.138	8.58

Table 7.2: Split-level performance metrics for the estimator.

in size, the fifth split continued to exhibit comparatively wide intervals, indicating greater uncertainty in the estimation of deeper splits in the tree.

The overall relationship between split order and split coverage is shown in Figure 7.1. As can be seen in the figure, a general decline in coverage as the split order increases was observed. This decline is particularly noticeable at smaller sample sizes. For example, when $n = 50$, coverage decreases from approximately 54% for the first split to below 5% for the fourth split. Larger sample sizes improve coverage for intermediate splits, especially for the second through fourth splits. As was mentioned previously, coverage of the fifth split remained very unlikely for all sample sizes.

In contrast, Figure 7.2 presents the average confidence interval widths by split order. The figure demonstrates that interval widths are largest for smaller sample sizes and generally become smaller as the expected sample size increases. In addition, as can be clearly seen in the figure, deeper splits in the tree have wider intervals than earlier splits. This effect is particularly noticeable for smaller sample sizes. The fifth split especially exhibits much larger interval widths. This reflects the increased variability associated with estimating lower-level splits in the tree structure.

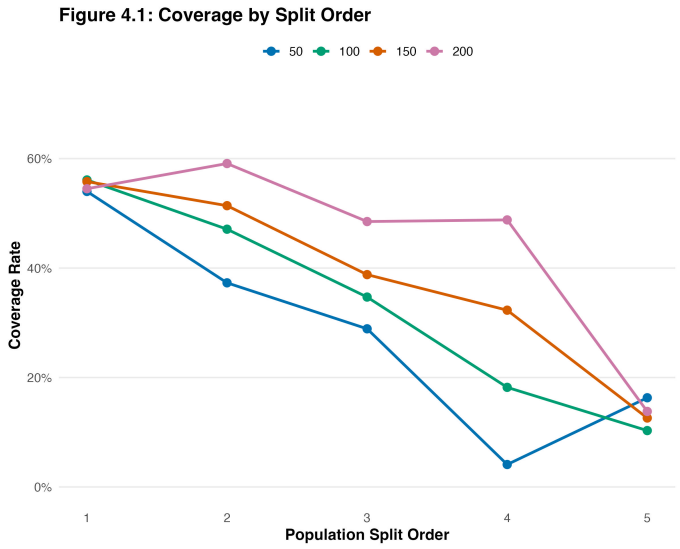


Figure 7.1: Coverage rate by population split order for the estimator across sample sizes.

The overall coverage results are presented in Table 7.3. Mean split coverage increased steadily with sample size, from 0.281 at $n = 50$ to 0.449 at $n = 200$. This represents the overall proportion of splits covered by the simulations at each sample size. Full region coverage, likely the most important result presented by this thesis, also improved with larger samples. It increased from a low of 0.293 at $n = 50$ to 0.349 at $n = 200$. In contrast, full split coverage (correct coverage of every split, in order, by the estimator) remained very rare throughout the simulation study. Some very limited improvement was observed for larger sample sizes, but it is clear that the probability of covering every split correctly and in order is small. At $n = 200$, only 5.3% of simulated trees covered all population splits simultaneously.

Sample Size	Mean Split Coverage	Full Region Coverage	Full Split Coverage
50	0.281	0.293	0.001
100	0.333	0.234	0.001
150	0.382	0.267	0.019
200	0.449	0.349	0.053

Table 7.3: Overall coverage metrics for the estimator.

Interestingly, it can be seen that the estimator $\hat{M}$ obtained higher region coverage at $n = 50$ than at both $n = 100$ and $n = 150$. This is not entirely unexpected. As Figure 7.2 shows, the confidence intervals at $n = 50$ were

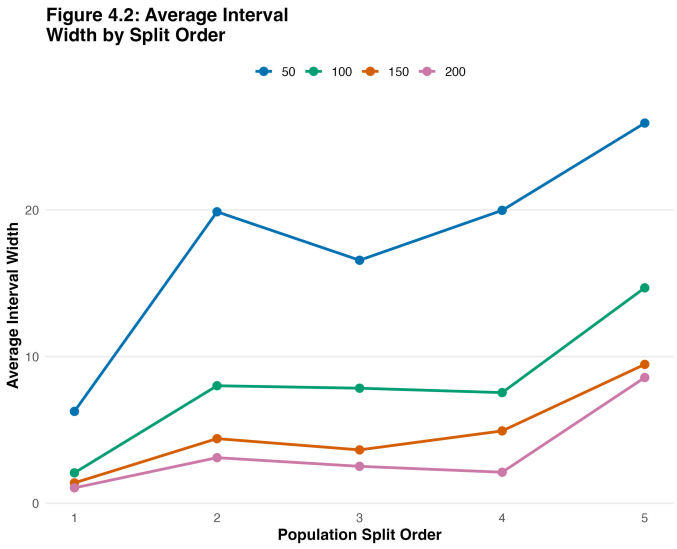


Figure 7.2: Average confidence interval width by population split order for the estimator across sample sizes.

considerably larger than at other sample sizes. This means that the "confidence boxes" constructed from those intervals were also much larger and highlights the fact that this method for approximating the coverage of the population parameter M through $\hat{M}$ does not penalize large intervals and is only concerned with coverage within those intervals of the population regions.

The relationship between sample size and overall tree and split coverage is shown in Figure 7.3. The overall trend suggests improved region recovery for larger sample sizes, although the relationship is not strictly monotone because larger intervals at smaller sample sizes which are less powerful can nevertheless have larger containment. In contrast to this, full split coverage remains substantially lower for all sample sizes.

When taken as a whole, the results indicate that the performance of the estimator $\hat{M}$ improves as the expected sample size increases. Confidence intervals become narrower while split coverage and region coverage generally increase. However, the results also show that lower-level splits are substantially more difficult to cover than earlier splits in the tree. This suggests that uncertainty accumulates as the recursive partitioning process progresses deeper into the tree structure.

The contrast between split-level coverage and full tree coverage is also notable. Individual splits can often be covered even when the complete population tree is not covered. In particular, the coverage of all population

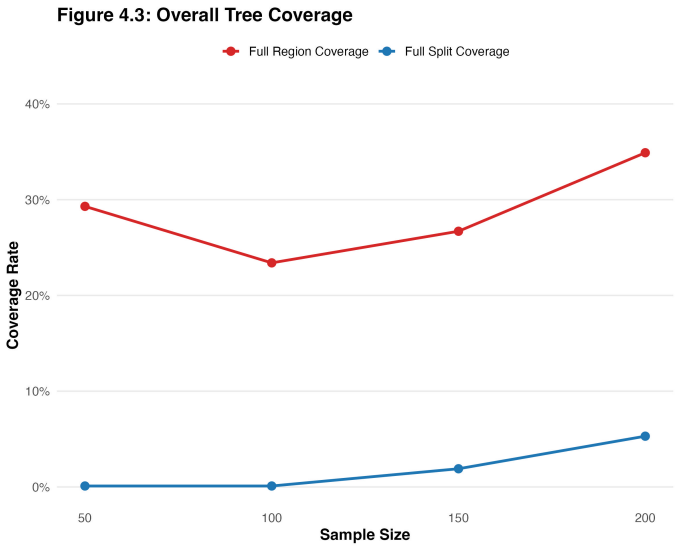


Figure 7.3: Overall tree coverage rates across sample sizes for the estimator.

splits simultaneously remains very uncommon, even at the largest sample sizes considered. This reflects the cumulative difficulty of accurately estimating every component of the tree structure simultaneously. However, it should be noted that region coverage showed better results. This indicates that, even if not all the splits were covered in the exact same fashion, the regional "confidence boxes" were still successfully able to capture the tree and its resulting structure even when coverage of all splits was highly unlikely.

8

Discussion

8.1 Analysis of the Results

The results presented above indicate that the estimator $\hat{M}$ is capable of recovering some of the structural features of the population tree M under repeated sampling, although the observed coverage rates were substantially lower than initially expected. In that sense, the simulation results suggest that it is possible to estimate regression trees as finite-population objects under design-based inference using the proposed estimator by constructing an RSS estimator based on Horvitz-Thompson estimators. This provides evidence supporting the main question motivating the thesis and implies that the proposed framework can successfully recover structural features of finite-population regression trees. Furthermore, the simulation results suggest that quantile-based confidence intervals can be used to construct approximate confidence regions for the population tree.

As was briefly mentioned in Chapter 7, the performance of the proposed estimator generally improved as the expected sample size increased. In particular, mean split coverage and full region coverage both tended to increase for larger samples, while the average widths of the confidence intervals became smaller. This behavior is broadly consistent with the design-based framework used in this thesis, since larger samples provide more information about the finite population and therefore allow the estimator to more closely approximate the population tree M.

At the same time, the results also show that deeper splits are considerably more difficult to cover, even at larger sample sizes. This result indicates that uncertainty increases throughout the recursive partitioning process as the effective sample size within each region decreases. Additionally, because splits depend on previous splits, an error near the beginning of the tree can propagate throughout the tree. The findings are consistent with prior work on tree based methods for survey data, where informative sampling and instability in recursive partitioning procedures have been identified as significant challenges [2, 13].

The relatively modest region coverage rates may also reflect a broader limitation of single-tree methods, which are known to exhibit high variance and instability under small perturbations of the data [2]. Exact recovery of full tree structure may therefore be inherently difficult even when higher-level splits are correctly identified. This sensitivity has been widely noted in the tree based methods literature [2, 3].

On the other hand, the results suggest that the proposed confidence interval procedure is able to capture at least part of the variability associated with the split selection process. Although the intervals were derived from an approximation based on quantile estimation and therefore do not account for the full variability produced by using CART, the estimator was still able to obtain relatively high region coverage when compared to deeper split coverage. These results indicate that the estimator is somewhat able to capture the structure of the population tree.

There also exists an important distinction between coverage and the size of the confidence intervals and boxes within the proposed framework. The method developed in this thesis prioritizes conservative containment of the population tree structure rather than tight estimation of the split locations themselves. As such, the comparatively high region coverage obtained at very low sample sizes (0.293 at $n = 50$) does not necessarily imply there was an accurate recovery of the exact tree structure. This is especially true in cases when the confidence regions are very wide.

The proposed measure for uncertainty and coverage is probably the most novel contribution of this thesis. Previous research highlights that the largest challenge in regards to using tree based methods in survey contexts is capturing uncertainty. Indeed, this is one of the motivating factors that led to Toth et al., (2025) using a Bayesian framework for analysis, as it enabled the construction of a measure of uncertainty [14]. The results obtained by this thesis suggest that it may be possible to construct a similar measure of uncertainty outside of a Bayesian framework.

Although Toth et al. (2025) use a Bayesian superpopulation framework instead of the finite-population design-based framework used in this thesis, both studies emphasize the importance of incorporating sampling weights directly into tree based estimation procedures under informative sampling designs [14]. Toth et al. demonstrated that accounting for the sampling design through weighted pseudo-likelihood methods allowed Bayesian tree models to produce prediction intervals with coverage rates close to their nominal targets while also quantifying uncertainty in the tree structure itself [14]. In contrast to their focus on posterior prediction and credible interval performance, this thesis evaluates the recovery of the finite-population tree structure through split and regional coverage. Despite their differences, both studies suggest that incorporating survey weights into recursive partitioning procedures is helpful when working within survey contexts and that meaningful measures of uncertainty for regression trees can be constructed for tree estimators in this context.

8.2 Strengths and Limitations of the Study

This thesis has both significant strengths and weaknesses. The first of these strengths is the innovative approach it adopts in regards to proposing a design-based framework for estimating regression trees under probability sampling. As has been mentioned before, though regression trees are widely used, no widely accepted methodology exists for their use with survey samples. This thesis contributes towards that goal. The methodology developed in this study treats the regression tree as a finite-population object and explicitly incorporates sampling weights through Horvitz–Thompson estimation. This may represent an important conceptual contribution because it extends recursive partitioning methods into a classical survey-sampling framework.

The use of the Horvitz-Thompson estimator is a significant strength because it means that the proposed estimator is highly general. While only Poisson sampling was used during the empirical evaluation of the method, it is relatively trivial to extend the framework developed to other sampling methods, including the more complicated sampling designs frequently used in practice. This ability to generalize increases the potential applicability of the methodology beyond the specific simulations presented in this thesis.

Another important strength is the proposed approach for conceptualizing uncertainty in regression trees. As has been noted before, one of the

main difficulties in applying tree based methods to survey data is the absence of a clear framework for measuring uncertainty in the tree structure itself. This thesis interprets split points as empirical quantiles and adapts Woodruff-style confidence interval methods for the splits. Through this technique, the thesis proposes a practical and interpretable way to approximate uncertainty around split locations. This enables the construction of confidence "boxes" which can be used to evaluate region coverage and indeed population tree coverage. Although this method is approximate, the approach provides a concrete and practical method for evaluating the stability of estimated trees under repeated sampling. In this sense, the thesis contributes not only an estimator for regression trees, but also a framework for assessing the reliability of the resulting trees.

Despite these strengths, it should be noted the method is clearly not working as expected: the coverage rate is considerably below the nominal 95% that was desired. This is likely the most significant weakness in this thesis, and could be explained by a number of factors. For example, it is pertinent to highlight that the proposed confidence intervals are only approximate. The intervals rely on interpreting split points as quantiles of the predictor distribution and therefore do not fully capture the adaptive nature of the CART algorithm. In particular, the approximation does not account for uncertainty arising from the selection of the splitting variable itself or from the recursive dependence between splits. Consequently, the intervals should not be interpreted as exact design-based confidence intervals, but rather as measures of relative stability and uncertainty. This can be clearly seen in the fact that, despite building 95% confidence intervals, the estimator did not cover 95% of splits or regions.

Another limitation concerns the empirical evaluation itself. The simulations were conducted using only the Boston Housing Dataset, which is relatively small and contains a limited number of observations. While the dataset is widely used and convenient for methodological research, it may not fully reflect the complexity of real survey data. In particular, the performance of the estimator under high-dimensional settings, strongly informative sampling designs, or highly heterogeneous populations remains unknown. The results obtained in this thesis should therefore be interpreted as exploratory and a starting point for further research.

Similarly, the evaluation considered only one sampling design, namely Poisson sampling, with inclusion probabilities proportional to the response variable. Although the estimator is theoretically applicable under more complex designs, the behavior of the method under stratified, clustered, or

multi-stage sampling was not investigated. Additional research is therefore needed to assess the robustness of the proposed framework under more realistic survey conditions.

The thesis is also limited by its focus on relatively shallow trees. Restricting the analysis to a small number of splits simplified interpretation and reduced computational complexity, but deeper trees may exhibit substantially greater instability and variability. Future work should therefore investigate how the proposed methods behave as tree complexity increases.

An additional limitation is that the thesis does not evaluate the bias of the point estimator $\hat{M}$. While the study focused primarily on coverage and uncertainty, analyzing the finite-sample bias of the estimated split points and tree structure would provide additional insight into the statistical properties of the estimator.

Finally, it should be further emphasized that the results presented in this thesis should be interpreted cautiously given the limitations of the simulation method. The confidence intervals proposed in this thesis are approximations and are based on a first-order interpretation of the split points as empirical quantiles. Furthermore, the Boston Housing Dataset is relatively small. Despite these limitations, the simulations provide evidence that the proposed estimator is capable of recovering some structural characteristics of the population regression tree under repeated sampling.

8.3 Future Areas for Research

There are some clear ways the method and evaluation of the method developed in this thesis can be extended in new research. The most obvious of these would be to evaluate the same method proposed using different sampling methods and different databases. Evaluating the estimator under these conditions would provide a clearer understanding of its practical applicability and robustness. A natural extension of this would also be to evaluate the estimator using different parameters (such as a different maximum number of regions J or a different minimum node size).

Another alternative area for future research would be to extend the methodology presented to other tree based methods. In particular, this thesis only focuses on regression trees but a natural extension would be to consider classification trees or ensemble methods (such as bagging or boosting) within a survey context. Extending the design-based framework to classification settings would likely require adapting both the splitting criteria and the

uncertainty measures. In a similar vein, ensemble methods such as bagging, random forests, and boosting represent particularly important directions for future work. These methods are generally more stable and have lower variance than single CART models. Incorporating sampling weights and design-based inference into ensemble methods could therefore combine the flexibility and predictive performance of modern machine learning methods with the inferential strengths of survey sampling theory.

Future work could also investigate alternative ways of conceptualizing distance and similarity between regression trees. This thesis focused primarily on split and region coverage because no widely accepted framework currently exists for measuring the distance between trees in a design-based context. However, other approaches are possible, particularly regarding predictive agreement with the population tree. It may be especially important to develop a more formal notion of distance between trees. This could provide a stronger theoretical foundation for measuring variability and constructing uncertainty measures for recursive partitioning methods.

The confidence intervals proposed in this thesis rely on a first-order approximation that interprets split points as empirical quantiles. While the simulation results suggest that this approach captures part of the variability associated with split selection, it does not fully account for the adaptive and recursive nature of CART. Future research could therefore focus on deriving asymptotic properties for the split estimators or developing alternative variance estimation procedures that better capture the uncertainty introduced by recursive partitioning.

Finally, future research could focus on establishing stronger theoretical results for the estimator proposed in this thesis. While the simulation results suggest that the estimator behaves reasonably well under repeated sampling, many of its formal statistical properties remain unknown. In particular, additional work is needed to investigate issues such as consistency of the estimated tree structure, asymptotic behavior of the split estimators, convergence of the induced partitions, and the theoretical properties of the proposed confidence intervals.

8.4 Conclusion

This thesis proposes an estimator $\hat{M}$ which can be used to estimate population regression trees from a sample. It also proposes a conceptual and practical way to compare regression trees to each other and measure coverage of the population tree M through $\hat{M}$. In doing so, the thesis extends

regression tree methodology into a finite-population, design-based framework by incorporating Horvitz–Thompson estimation directly into the recursive partitioning process. The findings highlight both the instability of regression trees and the difficulties of estimating them using survey data. They also provide evidence that the estimator can recover some aspects of the population tree while incorporating sampling probabilities directly into the recursive partitioning process. This method for the evaluation of the coverage of the population tree through the regions constructed from the confidence intervals is the thesis' most innovative contribution.

The framework developed here provides a foundation for future research on design-based recursive partitioning methods and survey-weighted tree estimation. More broadly, the thesis demonstrates that finite-population regression trees can be conceptualized as estimable population parameters and that their uncertainty can be studied using tools derived from survey sampling theory. The methodology developed is an initial step towards building a reliable methodology for estimating regression trees using samples. Considerable additional theoretical and empirical work is still needed to better understand the statistical properties of the proposed estimator, improve the proposed uncertainty measures, and extend the framework to more complex sampling designs and tree based methods. Nevertheless, the results presented here suggest that combining recursive partitioning with design-based inference is both feasible and promising.

Bibliography

[1] Naomi Altman and Martin Krzywinski. “Ensemble methods: bagging and random forests”. en. In: *Nature Methods* 14.10 (Oct. 2017), pp. 933–934. ISSN: 1548-7105. DOI: 10.1038/nmeth.4438. URL: https://www.nature.com/articles/nmeth.4438 (visited on 02/16/2026).

[2] Diya Bhaduri, Daniell Toth, and Scott H. Holan. “A Review of Tree-Based Methods for Analyzing Survey Data”. en. In: *WIREs Computational Statistics* 17.1 (2025). _eprint: https://wires.onlinelibrary.wiley.com/doi/pdf/10.1002/v e70010. ISSN: 1939-0068. DOI: 10.1002/wics.70010. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/wics.70010 (visited on 01/21/2026).

[3] Leo Breiman et al. *Classification and Regression Trees*. 1st ed. New York: Chapman and Hall/CRC, 1984. ISBN: 978-1-315-13947-0. DOI: 10.1201/9781315139470.

[4] Trent D. Buskirk. “Surveying the Forests and Sampling the Trees: An overview of Classification and Regression Trees and Random Forests with applications in Survey Research”. en. In: *Survey Practice* 11.1 (Jan. 2018). DOI: 10.29115/SP-2018-0003. URL: https://www.surveypractice.org/article/2709-surveying-the-forests-and-sampling-the-trees-an-overview-of-classification-and-regression-trees-and-random-forests-with-applications-in-survey-resear (visited on 01/21/2026).

[5] Mehdi Dagdoug, Camelia Goga, and David Haziza. “Model-Assisted Estimation Through Random Forests in Finite Population Sampling”. In: *Journal of the American Statistical Association* 118.542 (Apr. 2023). _eprint: https://doi.org/10.1080/01621459.2021.1987250, pp. 1234–1251. ISSN: 0162-1459. DOI: 10.1080/01621459.2021.1987250. URL: https://doi.org/10.1080/01621459.2021.1987250 (visited on 02/16/2026).

[6] David G. T. Denison, Bani K. Mallik, and Adrian F. M. Smith. “A Bayesian CART algorithm”. In: *Biometrika* 85.2 (June 1998), pp. 363–377. ISSN: 0006-3444. DOI: 10.1093/biomet/85.2.363. URL: https://doi.org/10.1093/biomet/85.2.363 (visited on 02/16/2026).

[7] Donald P. Green and Holger L. Kern. “Modeling Heterogeneous Treatment Effects in Survey Experiments with Bayesian Additive Regression Trees”. In: *Public Opinion Quarterly* 76.3 (Jan. 2012), pp. 491–511. ISSN: 0033-362X. DOI: 10.1093/poq/nfs036. URL: https://doi.org/10.1093/poq/nfs036 (visited on 02/16/2026).

[8] Christoph Kern, Thomas Klausch, and Frauke Kreuter. “Tree-based Machine Learning Methods for Survey Research”. In: *Survey research methods* 13.1 (Apr. 2019), pp. 73–93. ISSN: 1864-3361. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC7425836/ (visited on 02/05/2026).

[9] Wei-Yin Loh et al. “Classification and Regression Trees and Forests for Incomplete Data from Sample Surveys”. In: *Statistica Sinica* 29.1 (2019), pp. 431–453. ISSN: 1017-0405. URL: https://www.jstor.org/stable/26563262 (visited on 02/16/2026).

[10] Malte Nalenz, Julian Rodemann, and Thomas Augustin. “Learning de-biased regression trees and forests from complex samples”. en. In: *Machine Learning* 113.6 (June 2024), pp. 3379–3398. ISSN: 1573-0565. DOI: 10.1007/s10994-023-06439-1. URL: https://doi.org/10.1007/s10994-023-06439-1 (visited on 02/16/2026).

[11] Polly Phipps and Daniell Toth. “Analyzing Establishment Nonresponse Using an Interpretable Regression Tree Model with Linked Administrative Data”. In: *The Annals of Applied Statistics* 6.2 (2012), pp. 772–794. ISSN: 1932-6157. URL: https://www.jstor.org/stable/41713473 (visited on 02/16/2026).

[12] Carl-Erik Särndal, Bengt Swensson, and Jan Wretman. *Model Assisted Survey Sampling*. en. Google-Books-ID: ufdONK3E1TcC. Springer Science & Business Media, Oct. 2003. ISBN: 978-0-387-40620-6.

[13] Daniell Toth and John Eltinge. *Building Consistent Regression Trees From Complex Sample Data*. arXiv:1407.1079 [stat]. July 2014. DOI: 10.48550/arXiv.1407.1079. URL: http://arxiv.org/abs/1407.1079 (visited on 02/16/2026).

[14] Daniell Toth, Scott H Holan, and Diya Bhaduri. “Bayesian Tree Models for Survey Sample Data”. In: *Journal of Survey Statistics and Methodology* 13.4 (Sept. 2025), pp. 445–464. ISSN: 2325-0992. DOI: 10.1093/j

ssam/smae050. URL: https://doi.org/10.1093/jssam/smae050 (visited on 02/16/2026).

[15] Daniel K. Yang and Daniell S. Toth. "Analyzing the Association of Objective Burden Measures to Perceived Burden with Regression Trees". EN. In: *Journal of Official Statistics* 38.4 (Dec. 2022), pp. 1125–1144. ISSN: 0282-423X. DOI: 10.2478/jos-2022-0048. URL: https://journals.sagepub.com/action/showAbstract (visited on 02/16/2026).

Appendix 1 — R Code

```
library(MASS)
library(tidyverse)
library(purrr)
library(scales)

############ SECTION 1: FUNCTIONS ----
#### Horvitz-Thompson ----
ht_mean <- function(y, w) {

  if (is.null(w)) w <- rep(1, length(y))

  sum(w * y) / sum(w)

}

ht_rss <- function(y, w, groups) {

  if (is.null(w)) w <- rep(1, length(y))

  mu <- tapply(seq_along(y), groups, function(idx) {

    sum(w[idx] * y[idx]) / sum(w[idx])

  })

  mu_vec <- mu[as.character(groups)]
  sum(w * (y - mu_vec)^2)

}
```

```
#### Tree ----
build_tree_ht <- function(y, X, max_splits, values, w = NULL, min_size = 1) {

  if (is.null(w)) w <- rep(1, length(y))

  N <- nrow(X)
  P <- ncol(X)

  groups <- rep(1, N)
  leaves <- c(1)
  splits <- list()

  for (it in 1:max_splits) {

    best <- list(rss = Inf)

    for (g in leaves) {

      idx_g <- which(groups == g)
      if (length(idx_g) < min_size) next

      for (j in 1:P) {

        xg <- X[[j]][idx_g]
        ux <- sort(unique(xg))
        if (length(ux) < 2) next

        candidates <- ux[-length(ux)]

        for (t in candidates) {

          left_idx  <- idx_g[xg <= t]
          right_idx <- idx_g[xg > t]

          if (length(left_idx) == 0 || length(right_idx) == 0) next

          temp_groups <- groups
          new_left  <- max(groups) + 1
```

```
        new_right <- max(groups) + 2

        temp_groups[left_idx]  <- new_left
        temp_groups[right_idx] <- new_right

        rss <- ht_rss(y, w, temp_groups)

        if (rss < best$rss) {
          best <- list(rss = rss,
                       var = j,
                       cut = t,
                       group = g,
                       left_id = new_left,
                       right_id = new_right)

        }
      }
    }
  }

  if (is.infinite(best$rss)) break

  g <- best$group
  idx_g <- which(groups == g)
  xg <- X[[best$var]][idx_g]

  left_idx  <- idx_g[xg <= best$cut]
  right_idx <- idx_g[xg > best$cut]

  groups[left_idx]  <- best$left_id
  groups[right_idx] <- best$right_id

  leaves <- setdiff(leaves, g)
  leaves <- c(leaves, best$left_id, best$right_id)

  splits[[it]] <- best

}
```

```
  predictions <- tapply(seq_along(y), groups, function(idx) {

    sum(w[idx] * y[idx]) / sum(w[idx])

  })

  if (length(splits) > 0) {

    split_df <- tibble(step = seq_along(splits),
                       variable = names(X)[sapply(splits, function(s) s$var)],
                       split = sapply(splits, function(s) s$cut),
                       group_split = sapply(splits, function(s) s$group),
                       left_child = sapply(splits, function(s) s$left_id),
                       right_child = sapply(splits, function(s) s$right_id))

  } else {

    split_df <- tibble()

  }

  list(splits = split_df,
       groups = groups,
       predictions = predictions,
       leaves = leaves)

}

#### Variance tools ----
weighted_cdf <- function(x, w, t) {

  if (is.null(w)) {

    w <- rep(1, nrow(x))

  }

  sum(w[x <= t]) / sum(w)
```

```
}

weighted_quantile <- function(x, w, p, upp_low) {

  if (is.null(w)) {

    w <- rep(1, nrow(x))

  }

  if (p <= 0) return(min(x))
  if (p >= 1) return(max(x))

  ord <- order(x)
  x <- x[ord]
  w <- w[ord]

  cum_w <- cumsum(w) / sum(w)

  if (upp_low == "lower") {

    idx <- which(cum_w <= p)

    if (length(idx) == 0) {

      return(min(x))

    }

    return(x[max(idx)])

  } else if (upp_low == "upper") {

    idx <- which(cum_w >= p)

    if (length(idx) == 0) {

      return(max(x))
```

```
    }

    return(x[min(idx)])

  }

}

var_cdf_poisson <- function(x, w, t) {

  if (is.null(w)) {

    w <- rep(1, nrow(x))

  }

  N_hat <- sum(w)
  z <- as.numeric(x <= t)

  p_hat <- sum(w * z) / N_hat

  var_est <- sum((w^2 - w) * (z - p_hat)^2) / (N_hat^2)

  return(var_est)

}

#### CI ----
compute_split_ci_recursive <- function(tree, X, w, alpha = 0.05/5) {

  if (is.null(w)) {

    w <- rep(1, nrow(X))

  }

  n <- sum(1/w)

  splits <- tree$splits
```

```
groups <- rep(1, nrow(X))

results <- splits %>%
  mutate(lower = NA_real_, upper = NA_real_)

z_alpha <- qt((1 - alpha / 2), df = n - 1)

for (i in seq_len(nrow(splits))) {

  var <- splits$variable[i]
  t_hat <- splits$split[i]
  g <- splits$group_split[i]

  idx <- groups == g

  x_region <- X[[var]][idx]
  w_region <- w[idx]

  p_hat <- weighted_cdf(x_region, w_region, t_hat)
  se <- sqrt(var_cdf_poisson(x_region, w_region, t_hat))

  p_L <- max(0, p_hat - z_alpha * se)
  p_U <- min(1, p_hat + z_alpha * se)

  t_L <- weighted_quantile(x_region, w_region, p_L, "lower")
  t_U <- weighted_quantile(x_region, w_region, p_U, "upper")

  results$lower[i] <- t_L
  results$upper[i] <- t_U

  left_id  <- max(groups) + 1
  right_id <- max(groups) + 2

  left_idx  <- idx & (X[[var]] <= t_hat)
  right_idx <- idx & (X[[var]] > t_hat)

  groups[left_idx]  <- left_id
  groups[right_idx] <- right_id
```

```
  }

  results

}

match_closest_splits <- function(pop_splits, est_splits) {

  matched <- list()

  used_pop <- list()

  for (i in seq_len(nrow(est_splits))) {

    v <- est_splits$variable_hat[i]
    est_split <- est_splits$split_hat[i]

    pop_v <- pop_splits %>%
      filter(variable == v)

    if (nrow(pop_v) == 0) {

      matched[[length(matched) + 1]] <-
        tibble(pop_step = NA_integer_,
               est_step = i,
               variable = NA_character_,
               variable_hat = v,
               split_pop = NA_real_,
               split_hat = est_split,
               lower = est_splits$lower[i],
               upper = est_splits$upper[i],
               matched = FALSE,
               covered = FALSE,
               width = est_splits$upper[i] - est_splits$lower[i])

      next

    }
```

```
if (is.null(used_pop[[v]])) {

  used_pop[[v]] <- c()

}

available <- !(seq_len(nrow(pop_v)) %in% used_pop[[v]])

if (!any(available)) {

  matched[[length(matched) + 1]] <-
    tibble(pop_step = NA_integer_,
           est_step = i,
           variable = v,
           variable_hat = v,
           split_pop = NA_real_,
           split_hat = est_split,
           lower = est_splits$lower[i],
           upper = est_splits$upper[i],
           matched = FALSE,
           covered = FALSE,
           width = est_splits$upper[i] - est_splits$lower[i])

  next

}

dists <- abs(pop_v$split[available] - est_split)

best_local <- which.min(dists)

best_idx <- which(available)[best_local]

pop_match <- pop_v$split[best_idx]

matched[[length(matched) + 1]] <-
  tibble(pop_step = pop_v$step[best_idx],
         est_step = i,
         variable = v,
```

```
            variable_hat = v,
            split_pop = pop_match,
            split_hat = est_split,
            lower = est_splits$lower[i],
            upper = est_splits$upper[i],
            matched = TRUE,
            covered = (pop_match >= est_splits$lower[i] & pop_match <= est_splits$upp
            width = est_splits$upper[i] - est_splits$lower[i])

    used_pop[[v]] <- c(used_pop[[v]], best_idx)

  }

  bind_rows(matched)

}

match_ordered_splits <- function(pop_splits, est_splits) {

  K <- min(nrow(pop_splits), nrow(est_splits))

  out <- vector("list", K)

  for(i in seq_len(K)) {

    pop_var <- pop_splits$variable[i]
    est_var <- est_splits$variable_hat[i]

    same_var <- pop_var == est_var

    pop_split <- pop_splits$split[i]

    lower <- est_splits$lower[i]
    upper <- est_splits$upper[i]

    covered <- FALSE

    if(same_var) {
```

```
      covered <- (pop_split >= lower & pop_split <= upper)

    }

    out[[i]] <- tibble(pop_step = i,
                       est_step = i,
                       variable_pop = pop_var,
                       variable_hat = est_var,
                       same_variable = same_var,
                       split_pop = pop_split,
                       split_hat = est_splits$split_hat[i],
                       lower = lower,
                       upper = upper,
                       covered = covered,
                       width = upper - lower)

  }

  bind_rows(out)

}

build_population_regions <- function() {

  vars <- c("rm", "lstat", "dis")

  make_region <- function(rm_low = -Inf, rm_up = Inf, l_low = -Inf, l_up = Inf, d_low

    tibble(var   = vars,
           lower = c(rm_low, l_low, d_low),
           upper = c(rm_up,  l_up,  d_up))

  }

  list(R1 = make_region(rm_up = 6.939, l_up  = 14.370, d_up  = 1.3567),
       R2 = make_region(rm_up = 6.540, l_up  = 14.370, d_low = 1.3567),
       R3 = make_region(rm_low = 6.540, rm_up  = 6.939, l_up   = 14.370, d_low  = 1.35
       R4 = make_region(rm_up = 6.939,l_low = 14.370),
       R5 = make_region(rm_low = 6.939,rm_up  = 7.420),
```

```
       R6 = make_region(rm_low = 7.420))

}

compute_ci_region_bounds <- function(tree_splits, ci_table) {

  # Force all variables defining the population regions
  vars_all <- c("rm", "lstat", "dis")

  bounds <- list()

  # Root region
  bounds[["1"]] <- tibble(var = vars_all,
                          lower = rep(-Inf, length(vars_all)),
                          upper = rep( Inf, length(vars_all)))

  next_id <- 2

  for(i in seq_len(nrow(tree_splits))) {

    split_var <- tree_splits$variable[i]
    parent <- as.character(tree_splits$group_split[i])

    ci_low <- ci_table$lower[i]
    ci_up <- ci_table$upper[i]

    parent_region <- bounds[[parent]]

    # Left child: x <= split
    left_region <- parent_region

    left_region$upper[left_region$var == split_var] <-
      min(left_region$upper[left_region$var == split_var],ci_up)

    # Right child: x > split
    right_region <- parent_region

    right_region$lower[right_region$var == split_var] <-
      max(right_region$lower[right_region$var == split_var], ci_low)
```

```
    left_id  <- as.character(next_id)
    right_id <- as.character(next_id + 1)

    next_id <- next_id + 2

    bounds[[left_id]]  <- left_region
    bounds[[right_id]] <- right_region

    # Remove parent after splitting
    bounds[[parent]] <- NULL

  }

  bounds

}

check_region_containment <- function(pop_regions, est_regions) {

  covered <- logical(length(pop_regions))

  for(p in seq_along(pop_regions)) {

    pop_r <- pop_regions[[p]]

    match_found <- FALSE

    for(e in seq_along(est_regions)) {

      est_r <- est_regions[[e]]

      ok <- TRUE

      for(v in pop_r$var) {

        pop_low <- pop_r$lower[pop_r$var == v]
        pop_up  <- pop_r$upper[pop_r$var == v]
```

```
        est_low <- est_r$lower[est_r$var == v]
        est_up  <- est_r$upper[est_r$var == v]

        if(pop_low < est_low) ok <- FALSE
        if(pop_up  > est_up ) ok <- FALSE

      }

      if(ok) {

        match_found <- TRUE
        break

      }

    }

    covered[p] <- match_found

  }

  tibble(region = names(pop_regions),
         covered = covered)

}

#### Sampling ----
poisson_sample <- function(y, X, n) {

  N <- length(y)

  pi_k <- n * (y / sum(y))
  pi_k <- pmin(pi_k, 1)

  included <- rbinom(N, 1, pi_k)
  s <- which(included == 1)

  list(y = y[s],
       X = X[s, ],
```

```
       w = 1 / pi_k[s])

}

#### Functions for simulations----
# Function for a simulation
evaluate_simulation <- function(sim_id, y, X, values, pop_splits, alpha = 0.05/5, n =

  samp <- poisson_sample(y, X, n)

  tree_hat <- build_tree_ht(samp$y, samp$X, 5, values, w = samp$w)

  ci <- compute_split_ci_recursive(tree_hat, samp$X, samp$w, alpha)

  #### Check splits of CI
  ci <- ci %>%
    mutate(variable_hat = tree_hat$splits$variable,
           split_hat = tree_hat$splits$split,
           sim_id = sim_id) %>%
    select(-variable)

  pop_splits_eval <- pop_splits %>%
    mutate(sim_id = sim_id)

  split_data <- match_ordered_splits(pop_splits = pop_splits_eval,
                                     est_splits = ci) %>%
    mutate(sim_id = sim_id)

  split_step_results <- split_data %>%
    filter(!is.na(pop_step)) %>%
    group_by(pop_step) %>%
    summarise(covered = any(covered), .groups = "drop")

  all_splits_covered <- all(split_step_results$covered)

  #### Checks coverage of regions
  pop_regions <- build_population_regions()

  ci_regions <-
```

```
    compute_ci_region_bounds(tree_splits = tree_hat$splits,
                             ci_table = ci)

  region_results <-
    check_region_containment(pop_regions,
                             ci_regions)

  region_ok <- all(region_results$covered)

  sim_result <- tibble(sim_id = sim_id,
                       all_regions_covered = region_ok,
                       all_splits_covered = all_splits_covered)

  list(sim = sim_result,
       splits = split_data)

}

evaluate_simulation_no_weights <- function(sim_id, y, X, values, pop_splits, alpha = 0

  samp <- poisson_sample(y, X, n)

  tree_hat <- build_tree_ht(samp$y, samp$X, 5, values)

  ci <- compute_split_ci_recursive(tree_hat, samp$X, w = NULL, alpha = alpha)

  #### Check splits of CI
  ci <- ci %>%
    mutate(variable_hat = tree_hat$splits$variable,
           split_hat = tree_hat$splits$split,
           sim_id = sim_id) %>%
    select(-variable)

  pop_splits_eval <- pop_splits %>%
    mutate(sim_id = sim_id)

  split_data <- match_closest_splits(
    pop_splits = pop_splits_eval,
    est_splits = ci) %>%
```

```
    mutate(sim_id = sim_id)

  #### Checks coverage of regions
  pop_regions <- build_population_regions()

  ci_regions <-
    compute_ci_region_bounds(tree_splits = tree_hat$splits,
                             ci_table = ci)

  region_results <-
    check_region_containment(pop_regions,
                             ci_regions)

  region_ok <- all(region_results$covered)

  sim_result <- tibble(sim_id = sim_id, all_regions_covered = region_ok)

  list(sim = sim_result,
       splits = split_data)

}

# Function for running many simulations and building results base
run_simulation <- function(S = 1000, n = 200) {

  pop_splits <- tree_pop$splits %>%
    mutate(step = row_number()) %>%
    select(step, variable, split, group_split)

  results <- map(1:S, ~evaluate_simulation(.x, y, X, values, pop_splits, n = n))

  results <- compact(results)

  sim_df <- map_dfr(results, "sim")
  split_df <- map_dfr(results, "splits")

  list(simulation_level = sim_df,
       split_level = split_df)
```

```
}

run_simulation_no_weights <- function(S = 1000, n = 200) {

  pop_splits <- tree_pop$splits %>%
    select(variable, split, group_split)

  results <- map(1:S, ~evaluate_simulation_no_weights(.x, y, X, values, pop_splits, n

  results <- compact(results)

  sim_df <- map_dfr(results, "sim")
  split_df <- map_dfr(results, "splits")

  list(simulation_level = sim_df,
       split_level = split_df)

}

############ PART 2: RUN SIMULATION ----

# Build population tree
y <- Boston$medv
X <- Boston[, 1:13]

values <- map(X, ~sort(unique(.x)))

tree_pop <- build_tree_ht(y, X, 5, values)
pop_splits <- tree_pop$splits

# Life, the universe and everything
set.seed(42)

# Run simulations at n=200
out <- run_simulation(S = 1000, n = 200)

simulation_results_weighted_200 <- out$simulation_level
split_results_weighted_200 <- out$split_level
```

```
# Print out most relevant results
split_results_weighted_200 %>%
  group_by(pop_step) %>%
  summarise(coverage_rate = mean(covered),
            avg_width = mean(width))

simulation_results_weighted_200 %>%
  summarise(split_recovery = mean(all_splits_covered),
            region_recovery = mean(all_regions_covered))

split_results_weighted_200 %>% summarise(mean = mean(covered))

#split_results_weighted_200 %>% count(matched)

#split_results_weighted_200 %>% group_by(matched) %>% summarise(mean = mean(covered))

simulation_results_weighted_200 %>% summarise(mean = mean(all_regions_covered))

# Run simulations at n=150
out <- run_simulation(S = 1000, n = 150)

simulation_results_weighted_150 <- out$simulation_level
split_results_weighted_150 <- out$split_level

# Print out most relevant results
split_results_weighted_150 %>%
  group_by(pop_step) %>%
  summarise(coverage_rate = mean(covered),
            avg_width = mean(width))

simulation_results_weighted_150 %>%
  summarise(split_recovery = mean(all_splits_covered),
            region_recovery = mean(all_regions_covered))

split_results_weighted_150 %>% summarise(mean = mean(covered))

#split_results_weighted_150 %>% count(matched)

#split_results_weighted_150 %>% group_by(matched) %>% summarise(mean = mean(covered))
```

```
simulation_results_weighted_150 %>% summarise(mean = mean(all_regions_covered))

# Run simulations at n=100
out <- run_simulation(S = 1000, n = 100)

simulation_results_weighted_100 <- out$simulation_level
split_results_weighted_100 <- out$split_level

# Print out most relevant results
split_results_weighted_100 %>%
  group_by(pop_step) %>%
  summarise(coverage_rate = mean(covered),
            avg_width = mean(width))

simulation_results_weighted_100 %>%
  summarise(split_recovery = mean(all_splits_covered),
            region_recovery = mean(all_regions_covered))

split_results_weighted_100 %>% summarise(mean = mean(covered))

#split_results_weighted_100 %>% count(matched)

#split_results_weighted_100 %>% group_by(matched) %>% summarise(mean = mean(covered))

simulation_results_weighted_100 %>% summarise(mean = mean(all_regions_covered))

# Run simulations at n=50
out <- run_simulation(S = 1000, n = 50)

simulation_results_weighted_50 <- out$simulation_level
split_results_weighted_50 <- out$split_level

# Print out most relevant results
split_results_weighted_50 %>%
  group_by(pop_step) %>%
  summarise(coverage_rate = mean(covered),
            avg_width = mean(width))
```

```
simulation_results_weighted_50 %>%
  summarise(split_recovery = mean(all_splits_covered),
            region_recovery = mean(all_regions_covered))

split_results_weighted_50 %>% summarise(mean = mean(covered))

#split_results_weighted_50 %>% count(matched)

#split_results_weighted_50 %>% group_by(matched) %>% summarise(mean = mean(covered))

simulation_results_weighted_50 %>% summarise(mean = mean(all_regions_covered))

############ PART 3: GRAPHS ----
# Theme/colors for making nice graphs
pub_theme <- theme_minimal(base_size = 13) +
  theme(panel.grid.minor = element_blank(),
        panel.grid.major.x = element_blank(),
        legend.position = "top",
        legend.title = element_blank(),
        plot.title = element_text(face = "bold"),
        axis.title = element_text(face = "bold"),
        strip.text = element_text(face = "bold"))

sample_cols <- c("50"  = "#0072B2",
                 "100" = "#009E73",
                 "150" = "#D55E00",
                 "200" = "#CC79A7")

metric_cols <- c("split_recovery"  = "#1F77B4",
                 "region_recovery" = "#D62728")

# Plot 1: Split coverage
split_plot_data <- bind_rows(
  split_results_weighted_50  %>%
    group_by(pop_step) %>%
    summarise(coverage_rate = mean(covered),
              avg_width = mean(width)) %>%
    mutate(n = 50),
  split_results_weighted_100 %>%
```

```
    group_by(pop_step) %>%
    summarise(coverage_rate = mean(covered),
              avg_width = mean(width)) %>%
    mutate(n = 100),
  split_results_weighted_150 %>%
    group_by(pop_step) %>%
    summarise(coverage_rate = mean(covered),
              avg_width = mean(width)) %>%
    mutate(n = 150),
  split_results_weighted_200 %>%
    group_by(pop_step) %>%
    summarise(coverage_rate = mean(covered),
              avg_width = mean(width)) %>%
    mutate(n = 200))

ggplot(split_plot_data,
       aes(x = pop_step,
           y = coverage_rate,
           color = factor(n),
           group = factor(n))) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 3) +
  scale_y_continuous(labels = percent_format(accuracy = 1),
                     limits = c(0, 0.7)) +
  scale_x_continuous(breaks = 1:5) +
  scale_color_manual(values = sample_cols) +
  labs(title = "Figure 4.1: Coverage by Split Order",
       x = "Population Split Order",
       y = "Coverage Rate",
       color = "Sample Size") +
  pub_theme


ggsave("Graphs/Split_coverage.jpg")

# Plot 2: Interval width plot
ggplot(split_plot_data,
       aes(x = pop_step,
           y = avg_width,
```

```
          color = factor(n),
          group = factor(n))) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 3) +
  scale_x_continuous(breaks = 1:5) +
  scale_color_manual(values = sample_cols) +
  labs(title = "Figure 4.2: Average Interval\nWidth by Split Order",
       x = "Population Split Order",
       y = "Average Interval Width",
       color = "Sample Size") +
  pub_theme

ggsave("Graphs/Interval_width.jpg")

# Plot 3: Region coverage
overall_plot_data <- tibble(n = c(50, 100, 150, 200),
                            split_recovery = c(0.001, 0.001, 0.019, 0.053),
                            region_recovery = c(0.293, 0.234, 0.267, 0.349)) %>%
  pivot_longer(cols = c(split_recovery, region_recovery),
               names_to = "metric",
               values_to = "value")

ggplot(overall_plot_data,
       aes(x = n,
           y = value,
           color = metric)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 3) +
  scale_y_continuous(labels = percent_format(accuracy = 1),
                     limits = c(0, 0.4) ) +
  scale_x_continuous(breaks = c(50, 100, 150, 200)) +
  scale_color_manual(values = metric_cols,
                     labels = c("Full Region Coverage", "Full Split Coverage")) +
  labs(title = "Figure 4.3: Overall Tree Coverage",
       x = "Sample Size",
       y = "Coverage Rate",
       color = NULL,
       linetype = NULL) +
  pub_theme
```

```
ggsave("Graphs/Region_coverage.jpg")
```